Algorithm Engineering

Part I: The Memory Hierarchy & Sorting

- IO Model and Basics
  - RAM vs. Two-Level Model (M, B, N)
  - I/O Analysis: Scanning, Binary Search vs. B-Trees
  - Virtual Memory Probabilistic Analysis
- External Sorting and Permuting
  - Sorting vs. Permuting Bottlenecks
  - MergeSort: Binary vs. Multi-way $(N/B \log_{M/B} N/M)$
  - Run Generation: Snow Plow $(2M)$
  - Disk Striping (D disks)
- Quicksort and Selection
  - In-Memory: 3-way, Dual Pivot, Bounded
  - Selection: Randomized Linear Time
  - External: Multi-way Quicksort & Oversampling

Part II: Streaming & Sets

- Random Sampling and Streams
  - Disk Sampling (Known N)
  - Reservoir Sampling (Unknown N, inductive proof)
- Intersection Algorithms
  - Merge-based vs. Binary Search-based
  - Galloping (Doubling) Search
  - Interpolation Search $(O(\log \log N))$

Part III: Advanced Data Structures

- Randomized Dictionaries
  - Skip Lists: Levels, Coin flips, I/O issues
  - Treaps: Rotations, Split/Merge, 3-sided Range Query
  - Random BST Average Depth Proof
- String Sorting and Tries
  - String Sorting Lower Bound $(d + n \log n)$
  - Multi-key Quicksort

- Radix Sorts (LSD/MSD)
- Tries: TST, Patricia, Front Coding
- Two-Level Indexing (Front Coding + Tries)

Part IV: Indexing & Hashing

- Full Text Indexing
  - Suffix Arrays & Binary Search
  - LCP Array: Kasai's Algorithm ($O(N)$)
  - Suffix Trees from SA+LCP
  - Text Mining (Repeated Substrings, Distance Constraints)
- Hashing Protocols
  - Universal Hashing
  - Power of Two Choices
  - Cuckoo Hashing: Graph theory & $O(1)$ lookup
- Filters and Perfect Hashing
  - Minimal Ordered Perfect Hashing (MOPHF)
  - Bloom Filters: False positive derivation
  - Spectral Bloom Filters

Part V: Compression

- Data Compression
  - Entropy (H_0) & Kraft's Inequality
  - Huffman Coding (Canonical reconstruction)
  - Arithmetic Coding (Intervals & Correctness)
  - Dictionary Compression: LZ77 & LZSS
  - Integer Coding: Unary, Gamma, Delta, Variable Byte
- Rank and Select
- Problem definition
- Naive Solution vs Succint Requirements - Jacobson's Solution: Superblocks, Blocks, Popcount ($n + o(n)$ bits) - Compressed Solution: Rank/Select via Elias-Fano

Exam Questions|Exam Questions

Exercises|Exercises

IO Model and Basics

1. The Need for a New Model

The Von Neumann RAM Model

Historically, algorithm analysis has relied on the **Random Access Machine (RAM)** model (or Von Neumann model).

- **Assumptions:**
  - Infinite memory.
  - Uniform access time: Accessing any memory cell takes constant time $O(1)$.
  - Atomic operations: Arithmetic and logical operations take unit time.
- **Success:** It has been highly successful because it is simple and generally predictive for small datasets or legacy hardware where the CPU-Memory gap was negligible.

The Hardware Reality: The "Memory Wall"

Modern architectures violate the RAM assumptions due to the **Memory Hierarchy**. There is a significant gap (the "Memory Wall") between the speed of the CPU and the speed of the storage systems.

- **Internal Memory (Cache/RAM):** Fast, small, volatile. Access time in nanoseconds.
- **External Memory (HDD/SSD):** Slow, huge, persistent. Access time in milliseconds.

  The I/O Bottleneck The difference in speed between accessing registers/cache and accessing a disk is a factor of approximately 10^5 **to** 10^6 .

  If a CPU cycle is 1 second:

  - Cache access $\approx$ Seconds.
  - RAM access $\approx$ Minutes.
  - Disk access $\approx$ **Months**.

Because of this disparity, the number of CPU instructions executed is often less relevant than the number of times data is moved between disk and memory. We need a model that counts **I/Os** (Input/Output operations).

---

2. The Two-Level Memory Model

To capture the I/O bottleneck, we use the **External Memory Model** (or Disk Access Model / I/O Model). This abstracts the computer into two levels:

1. **Internal Memory:** Limited size M.
2. **External Memory:** Unlimited size.

Key Parameters

We define the system using three fundamental parameters:

- N: The size of the problem instance (number of items).
- M: The size of the **Internal Memory** (number of items that fit in RAM).
- B: The size of a **Disk Block** (number of items transferred in a single I/O).

Rules of the Model

1. Computation can only happen on data present in **Internal Memory**.
2. Data is transferred between Internal and External memory in **blocks** of size B.
3. **Cost Function:** The complexity of an algorithm is the total number of **I/O operations** (or "page faults") performed. We denote this as $\mathcal{C}_{IO}$.
   - CPU time is considered free (or secondary).

- We assume $1 \ll B \leq M < N$.

---

3. Basic I/O Analysis

3.1 Scanning (Linear Scan)

Consider the problem of summing an array A of N integers stored contiguously on disk.

Algorithm:

1. Read the first block of the array into memory.
2. Process all B items in that block.
3. Discard the block and read the next one.

Analysis:

- **RAM Model Cost:** $O(N)$ (we touch every item).
- **I/O Model Cost:** Since we read data in chunks of B, we perform one I/O for every B items.

$$\text{Cost}_{scan}(N) = \Theta\left(\frac{N}{B}\right) = \Theta\left(\lceil N/B \rceil\right) \text{ I/Os}$$

Note on Spatial Locality: The term N/B highlights the power of **spatial locality**. By grouping data physically close to each other, a single I/O retrieves B useful items, effectively dividing the cost by B. This is the theoretical lower bound for any algorithm that must read the entire input.

Generalized Scanning: The $A(s, b)$ Family We can generalize scanning to account for different access strides .

- Let b be a **logical block size** (smaller than physical block B).
- Let s be a **step size** (jump).
- Algorithm $A(s, b)$: Scan logical blocks A_j jumping by step s.

$$j = (i \cdot s) \mod (N/b)$$

I/O Complexity: The complexity depends on how many items we skip versus how many fit in a physical block B.

$$\text{Cost} = \frac{N}{B} \cdot \min\left(s, \frac{B}{b}\right)$$

- **Case 1 ($s \leq B/b$):** We skip small amounts. We still benefit partially from the block read, but we waste bandwidth. Cost $\approx s \cdot (N/B)$.
- **Case 2 ($s > B/b$):** We jump so far that every access requires a new block fetch. Cost $\approx N/b$ (effectively random access behavior).

3.2 Searching: Binary Search vs. B-Trees

Consider searching for a specific key k in a sorted array of size N.

Case A: Standard Binary Search In the RAM model, Binary Search is optimal with $O(\log_2 N)$ comparisons. In the I/O model, however, it performs poorly .

1. We look at the middle element. This causes **1 I/O**.
2. We jump to the middle of the left or right subarray. This jump likely lands in a different block, causing **1 I/O**.

3. This continues until the search interval is small enough to fit inside a single block ($\leq B$).

Cost Analysis: The number of levels in the recursion before the interval size drops to B is:

$$\text{Cost}_{binsearch}(N) = O\left(\log_2 \frac{N}{B}\right) \text{ I/Os}$$

Critique: The base of the logarithm is 2. For large N, this is still significant. We are not utilizing the block transfer B effectively; we load B items but only look at 1 (the pivot).

Case B: B-Tree Search (Multi-way Search) To optimize for B, we use a **B-Tree** (or generally, an (a, b)-tree).

- **Node Structure:** Each node is the size of a disk page (B).
- **Fan-out:** A node contains $\Theta(B)$ keys and $\Theta(B)$ pointers to children.

Algorithm:

1. Load the root node (1 I/O).
2. Perform binary search *in memory* over the $\Theta(B)$ keys within that node to find the correct child pointer (0 I/O cost, only CPU time).
3. Load the child node (1 I/O).

Cost Analysis: The height of the tree determines the I/O cost. Since the fan-out is $\Theta(B)$, the height is logarithmic in base B:

$$\text{Cost}_{B-Tree}(N) = O\left(\log_B \frac{N}{B}\right) \approx O\left(\log_B N\right) \text{ I/Os}$$

Note: $\log_B(N/B) = \log_B N - 1$.

Comparison:

$$\frac{\text{Binary Search}}{\text{B-Tree}} = \frac{\log_2 N}{\log_B N} = \frac{\ln N / \ln 2}{\ln N / \ln B} = \log_2 B$$

Since B is typically large (e.g., $B = 4096$ bytes ≈ 1000 integers), $\log_2 B \approx 10$. **B-Trees are a factor of $\log_2 B$ faster than Binary Search on disk.**

---

4. Virtual Memory: A Probabilistic Analysis

Why does the OS Virtual Memory system perform poorly when algorithms are not I/O aware? We can model this probabilistically to see the impact of the "penalty" factor.

The Setup

- Let $N = (1 + \epsilon)M$. The dataset is slightly larger than RAM.
- Let c be the **I/O penalty** (the ratio of Disk access time to RAM access time, e.g., $c \approx 10^5$).
- Let a be the probability that an instruction accesses memory (e.g., $a \approx 0.3$).
- Let $p(\epsilon)$ be the probability of a **Page Fault** (data is not in RAM).

Average Step Cost (T_{avg})

The average time to execute one step of the algorithm is the weighted sum of executing it in RAM versus fetching from Disk.

$$T_{avg} = \underbrace{(1-a)\cdot 1}_{\text{CPU ops}} + \underbrace{a \cdot [\underbrace{(1-p(\epsilon))\cdot 1}_{\text{Mem Hit}} + \underbrace{p(\epsilon)\cdot c}_{\text{Mem Miss (Fault)}}]}_{\text{Memory Access}}$$

Assuming c is very large, the term $p(\epsilon) \cdot c$ dominates. The formula simplifies to:

$$T_{avg} \approx a \cdot p(\epsilon) \cdot c$$

Interpretation

If we have a standard algorithm (like random access in a Hash Table or Binary Search) where we access memory randomly, $p(\epsilon)$ might be small, but non-negligible.

- **Scenario:** $a = 0.3$, $c = 10^5$.
- **Low Fault Rate:** Even if we only fault once every 1000 accesses ($p(\epsilon) = 10^{-3}$):

  $$T_{avg} \approx 0.3 \cdot 10^{-3} \cdot 10^5 = 30$$

  The algorithm runs **30 times slower** than CPU speed.
- **High Fault Rate:** If $p(\epsilon) \approx 1$ (thrashing), the algorithm runs 10^5 times slower.

This mathematical derivation confirms that minimizing $p(\epsilon)$—by designing algorithms that respect locality (B)—is crucial for performance, far more than optimizing CPU instructions.

External Sorting and Permuting

1. Sorting vs. Permuting in the I/O Model

In the standard RAM model, sorting takes $O(N \log N)$ while permuting (reordering data according to a given permutation π) takes linear time $O(N)$. In the **External Memory Model**, this intuition changes drastically.

1.1 The I/O Complexity

- **Sorting Cost:**

  $$\text{Sort}(N) = O\left(\frac{N}{B} \log_{M/B} \frac{N}{M}\right) \text{ I/Os}$$

- **Permuting Cost:**

  $$\text{Perm}(N) = O\left(\min\left\{N, \text{Sort}(N)\right\}\right) \text{ I/Os}$$

1.2 The Permuting Bottleneck

Permuting is considered an **I/O bottleneck**. * If we naively follow the permutation π to move elements one by one: 1. Read index i. 2. Fetch element from position $\pi[i]$. 3. If $\pi[i]$ is in a different block than the previous element, we pay **1 I/O**. 4. **Worst Case:** Every access is a cache miss. Total Cost = $O(N)$ I/Os. * **Lack of Spatial Locality:** Unlike scanning (N/B), we lose the block advantage because the target locations are scattered randomly across the disk.

Permuting via Sorting: The Algorithm We can reduce the permuting problem to sorting to achieve the lower I/O cost. Given vector S (data) and P (target positions), where $S[i]$ should move to position $P[i]$:

1. **Scan** S and write pairs $\langle \text{item}, \text{old_pos} \rangle$.
2. **Scan** P and write pairs $\langle \text{new_pos}, \text{old_pos} \rangle$.
3. **Sort** the P pairs by their second component (old_pos).
4. **Scan** the sorted P and S together (merge). Since they are both ordered by old_pos, we can link $\langle \text{item} \rangle$ with $\langle \text{new_pos} \rangle$. Create pairs $\langle \text{item}, \text{new_pos} \rangle$.
5. **Sort** these new pairs by their second component (new_pos).

6. **Scan** and extract just the items.

Total Cost: 4 Scans + 2 Sorts $\approx O(\text{Sort}(N))$.

---

2. MergeSort Analysis

MergeSort is the standard approach for external sorting. We divide the file into chunks that fit in memory, sort them (creating "runs"), and then merge them.

2.1 Binary MergeSort

This is the standard 2-way merge adapted for disk. 1. **Run Generation:** Create N/M sorted runs. 2. **Merging:** Merge 2 runs at a time.

Analysis: * Number of Passes (Recursion Depth): $\log_2(N/M)$. * Cost per Pass: We read and write the entire dataset once per level $\rightarrow 2 \cdot (N/B)$. * **Total Cost:**

$$\text{Cost}_{Binary} = O\left(\frac{N}{B}\log_2\frac{N}{M}\right) \text{ I/Os}$$

* *Critique:* The base 2 is inefficient. We are not using the available memory M to merge more than 2 streams at once.

2.2 Multi-way MergeSort (k-way Merge)

To optimize, we maximize the number of streams we merge simultaneously.

The Algorithm: 1. **Setup:** We have memory M and block size B. 2. **Fan-in:** We can hold 1 block from k different input streams and 1 block for the output stream in memory.

$$k \cdot B + 1 \cdot B \leq M \implies k \approx \frac{M}{B} - 1$$

3. **Process:** Use a **Min-Heap** of size k in memory to efficiently select the smallest item among the k streams. * CPU Time per item: $O(\log k)$. * I/O behavior: When an input buffer empties, load the next block (1 I/O). When the output buffer fills, write it to disk (1 I/O).

Rigorous Complexity Derivation: * **Initial Runs:** N/M runs (or $N/2M$ with Snow Plow). * **Reduction Factor:** In each pass, we reduce the number of runs by a factor of $k \approx M/B$. * **Number of Passes:**

$$\text{Passes} = \log_{M/B}\left(\frac{N}{M}\right)$$

* **Total Cost:**

$$\text{Cost}_{Multi-way} = O\left(\frac{N}{B}\log_{M/B}\frac{N}{M}\right) \text{ I/Os}$$

> **Why Base M/B?** The base M/B represents the **fan-in** of the merge process. We leverage the entire memory M to buffer blocks B. This is significantly faster than binary merge ($\log_2$ vs $\log_{M/B}$).

---

3. Run Generation: The "Snow Plow"

Instead of just reading M items, sorting them, and writing them out (producing runs of length M), we can use the "Snow Plow" technique to generate runs of average length $2M$.

3.1 The Analogy

Imagine a snow plow moving around a circular track (memory). Snow (data) falls continuously. The plow pushes out snow that fits the current order, but snow that falls "behind" the plow (smaller than current max) stays for the next round.

3.2 The Algorithm

We partition memory M into a **Min-Heap (H)** and an **Unsorted Buffer (U)**. Initially, H is full (size M), U is empty.

1. **Extract:** Remove min element m from H and write to Output.
2. **Read:** Read next element x from Input.
3. **Compare & Insert:**
   - **Case A ($x \geq m$):** x can be part of the *current* run. Insert x into H.
   - **Case B ($x < m$):** x is smaller than what we just wrote; it cannot be in the current sorted run. Place x into U (it is "dead" for now).
4. **Heap Shrinks:** If we put x in U, the effective size of H decreases.
5. **Restart:** When H is empty (memory is full of "dead" elements in U), the current run ends. Move all U to H, rebuild Heap, start new run.

3.3 Average Run Length Proof

- **Intuition:** Assume input is random.
- When we read an item x, $P(x < m) \approx 1/2$.
- So, for every 2 items we read, 1 goes to H (extending the run) and 1 goes to U (stored for next run).
- We start with M items. To fill U with M items (ending the phase), we must have processed roughly $2M$ items total.
- **Result:** Average run length $= 2M$.
- *Benefit:* Halves the number of initial runs, saving 1 merge pass in some cases.

---

4. Disk Striping (D Disks)

To increase bandwidth, we use D independent disks.

4.1 Striping Technique

- **Concept:** Distribute data cyclically across disks (Block 0 on Disk 0, Block 1 on Disk 1, etc.).
- **Logical Block:** We treat the D disks as a single large disk with a **Logical Block Size** of:

$$B' = D \times B$$

- **Memory Constraint:** We need enough memory to buffer one block from every disk:

$$M \geq D \times B$$

4.2 Complexity with Striping

We substitute B with $D \cdot B$ in the Multi-way MergeSort formula:

$$\text{Cost}_{Striped} = O\left(\frac{N}{D \cdot B} \log_{M/(D \cdot B)} \frac{N}{M}\right) \text{ I/Os}$$

4.3 Comparison vs Theoretical Lower Bound

The theoretical lower bound for sorting with D disks is slightly better than what Disk Striping achieves.

$$\text{Ratio} = \frac{\text{Disk Striping}}{\text{Lower Bound}} \approx \frac{\log(M/B)}{\log(M/B) - \log D}$$

* **Interpretation:** Disk Striping is asymptotically slower, but as $M \to \infty$, the ratio approaches 1. It is efficient enough for practical purposes.

---

5. Lower Bounds

Is Multi-way MergeSort optimal?

5.1 The Lower Bound Formula

The theoretical lower bound for sorting N items on D disks is:

$$\Omega\left(\frac{N}{D \cdot B} \log_{M/B} \frac{N}{D \cdot B}\right) \text{ I/Os}$$

5.2 Optimality Check

1. **Single Disk ($D = 1$):**
   - Algorithm: $\frac{N}{B} \log_{M/B} \frac{N}{M}$
   - Lower Bound: $\frac{N}{B} \log_{M/B} \frac{N}{B}$
   - *Verdict:* **Optimal.** (Since $\frac{N}{M}$ and $\frac{N}{B}$ are close enough inside the log).
2. **Multiple Disks ($D > 1$):**
   - Algorithm (Striping): Base is $M/(D \cdot B)$.
   - Lower Bound: Base is M/B.
   - *Verdict:* **NOT Optimal.** Disk Striping reduces the fan-out of the merge because the logical block B' consumes more memory slots.
   - *Note:* To achieve optimality for $D > 1$, complex algorithms like **GreedSort** are required (not covered in standard implementation).

Quicksort and Selection

1. In-Memory Quicksort Variants

While standard Quicksort is $O(N \log N)$ on average, optimizations are crucial for handling pathological cases (duplicates) and modern hardware (cache efficiency).

1.1 3-Way Partitioning (Bentley-McIlroy)

Standard Quicksort suffers when the input has many **duplicate keys**, potentially degrading to $O(N^2)$. The 3-way partition divides the array $S[i, j]$ into three segments relative to the pivot P:

1. **Less than P:** $S[i, l-1]$
2. **Equal to P:** $S[l, r-1]$
3. **Greater than P:** $S[r, c-1]$

Algorithm Logic:

- Use pointers to swap elements into their regions.

- **Crucial Optimization:** The central segment (elements equal to P) is **excluded** from recursive calls.
- If the array consists of all equal keys, this step runs in $O(N)$ and terminates immediately, whereas standard Quicksort would recurse indefinitely.

1.2 Dual-Pivot Quicksort (Yaroslavskiy)

The default algorithm in Java 7+ and Python. It uses **two pivots** (p, q with $p < q$) to partition the array into three regions:

1. $x < p$
2. $p \leq x \leq q$
3. $x > q$

Performance Paradox:

- Theoretically, Dual-Pivot requires **more comparisons** ($1.9N \ln N$) than single-pivot ($1N \ln N$).
- However, it is practically **faster** (often $> 10\%$).
- **Reason:** It minimizes **Branch Mispredictions** . Modern CPUs pipeline instructions; a random conditional jump (like in Quicksort partition) clears the pipeline. Dual-Pivot reduces the number of memory accesses and behaves better with CPU caches, making the "cost per comparison" lower.

1.3 Bounded Quicksort

Standard Quicksort is recursive. In the worst case (unbalanced partitions), the recursion depth can reach $O(N)$, causing a **Stack Overflow**.

The Solution: To guarantee $O(\log N)$ stack space, we rely on **Tail Recursion Elimination** on the larger sub-problem.

Pseudocode:

```
BoundedQS(S, i, j) {
    while (j - i > n0) { // Small subarrays use Insertion Sort
        // 1. Random Pivot Selection & Partitioning
        r = random(i, j);
        swap(S[i], S[r]);
        p = partition(S, i, j);

        // 2. Identify smaller half to recurse on
        if (p <= (i + j) / 2) {
            // Left side is smaller: Recurse Left
            BoundedQS(S, i, p - 1);
            i = p + 1; // Loop handles the Right side (Tail Call)
        } else {
            // Right side is smaller: Recurse Right
            BoundedQS(S, p + 1, j);
            j = p - 1; // Loop handles the Left side (Tail Call)
        }
    }
    InsertionSort(S, i, j);
}
```

Since we always recurse on the smaller part (at most half the size), the recursion depth cannot exceed $\log_2 N$.

1.4 Proof of Expected Comparisons

We prove that the expected number of comparisons is $O(N \log N)$ 4.

Let S' be the sorted version of the input S.

Define indicator variable $X_{u,v} = 1$ if $S'[u]$ is compared with $S'[v]$, and 0 otherwise (for $u < v$).

Total Comparisons:

$$C = \sum_{u=1}^{N-1} \sum_{v=u+1}^{N} X_{u,v}$$

By Linearity of Expectation:

$$E[C] = \sum_{u=1}^{N-1} \sum_{v=u+1}^{N} P(X_{u,v} = 1)$$

Probability Logic:

- $S'[u]$ and $S'[v]$ are compared **if and only if** one of them is selected as the pivot *before* any other element in the range $S'[u \dots v]$.
- If a pivot is chosen from strictly inside (u, v), then u and v will be separated into different partitions and never compared.
- The range $S'[u \dots v]$ has size $v - u + 1$.
- Since pivots are random, every element in the range has probability $\frac{1}{v-u+1}$ of being chosen first.
- We need either u or v to be chosen first:

$$P(X_{u,v} = 1) = \frac{2}{v - u + 1}$$

Summation:

$$E[C] = \sum_{u=1}^{N-1} \sum_{v=u+1}^{N} \frac{2}{v - u + 1} \leq \sum_{u=1}^{N} 2 \sum_{k=1}^{N} \frac{1}{k} \approx 2N \ln N$$

---

2. The Selection Problem Problem:

Find the k-th smallest element in an unsorted sequence S.

2.1 Complexity Landscape

- **Sorting:** Sort S and pick index k. Time: $O(N \log N)$.
- **QuickSelect:** Randomized partitioning. Time: $O(N)$ expected.

Heap-based Selection ($O(N \log k)$) Useful when k is small relative to N.

1. Maintain a **Max-Heap** of size k.
2. Fill it with the first k elements.
3. Scan the rest of the array ($N - k$ items).
4. If a new item $x <$ Heap.Max(), remove Max and insert x.
5. **Result:** The Max of the heap is the k-th smallest item.

2.2 Randomized Selection (QuickSelect)

Similar to Quicksort, but we only recurse on **one** side. **Algorithm:**

1. Pick a random pivot.
2. Partition S into $S_<, S_=, S_>$.
3. If $k \leq |S_<|$, recurse on $S_<$.
4. Else if $k \leq |S_<| + |S_=|$, return Pivot.
5. Else, recurse on $S_>$ seeking rank $k - (|S_<| + |S_=|)$. **Proof of Linear Expected Time:**

- A "Good Selection" occurs if the pivot lands in the middle third of the sorted sequence (ranks $[N/3, 2N/3]$). * This guarantees neither $|S_<|$ nor $|S_>|$ exceeds $2N/3$.
- Probability of Good Selection = 1/3.
- Recurrence for expected time $\hat{T}(N)$:

$$\hat{T}(N) \leq O(N) + \hat{T}\left(\frac{2N}{3}\right)$$

By the Master Theorem, this sums to a geometric series dominated by the first term:

$$\hat{T}(N) = O(N)$$

- **I/O Complexity:** Since partitioning is a scan, Cost = $O(N/B)$ I/Os.

---

3. Multi-way Quicksort (External Memory)

While Multi-way MergeSort is a **bottom-up** (merge) approach, Multi-way Quicksort is a **top-down** (distribution) approach6.

3.1 Algorithm Design

1. **Distribution:** Select $k-1$ pivots to divide the input range into k buckets (partitions)7.
2. **Scanning:** Read the input sequence. For every element, determine which bucket it belongs to and write it to the corresponding buffer 8.
   - **Constraints:** We need k output buffers (size B) in memory. Thus,

$$k \approx M/B$$

.

3. **Recursion:** Recursively sort each bucket. If a bucket fits in memory ($< M$), load it and sort it internally.

3.2 Pivot Selection via Oversampling

The critical flaw of Quicksort is unbalanced partitions. In external memory, a bad partition wastes entire I/O passes. We need "perfect" pivots that split N items into k buckets of size $\approx N/k$.

The Oversampling Technique 10:

1. Draw a random sample of size s from the dataset.
2. Sort the sample.
3. Pick elements at regular intervals (s/k) to be the $k-1$ pivots.

Theorem (Sample Size) 11:

To ensure that no bucket exceeds size $4N/k$ with probability $\geq 1/2$, we need an oversampling factor a such that:

$$a + 1 = \frac{1}{2} \ln k$$

Total sample size:

$$s = (a+1)k - 1 \approx \frac{k}{2} \ln k$$

Note: The sample size depends on k (number of buckets) and $\ln k$.

3.3 I/O Complexity Analysis

- **Fan-out:** $k \approx M/B$.
- **Cost per Level:** We read and write the whole dataset once: $2(N/B)$.
- **Number of Levels:** The recursion depth is $\log_k(N/M)$.

$$\text{Cost}_{MultiQuick} = O\left(\frac{N}{B}\log_{M/B}\frac{N}{M}\right) \text{ I/Os}$$

This matches the sorting lower bound.

3.4 Comparison: MergeSort vs Quicksort

Feature	Multi-way MergeSort	Multi-way Quicksort
Paradigm	Merging (Bottom-Up)	Distribution (Top-Down)
I/O Access	Sequential Read / Sequential Write	Sequential Read / **Random** Write (to k buffers)
Space	Easy to manage	Harder (need robust pivots)
Preference	Generally preferred for stability and guaranteed balance.	Preferred if random writes are fast or for parallel systems.

Random Sampling and Streams

Sampling is fundamental when datasets are too massive to process entirely. We analyze two scenarios: one where the dataset size N is known and stored on disk, and one where data arrives as a **Stream** of unknown length.

---

1. Disk Sampling (Known Length N)

1.1 Algorithm A: Iterative Scanning

We want to select exactly s items from a file of size N.

The Algorithm: Iterate through the file item by item (from $j = 0$ to $N - 1$). Let:

- t: number of items scanned so far.
- m: number of items *already selected* so far.
- s: total items we want to select.

For the current item x, we select it with probability: $$P(\text{select } x) = \frac{s - m}{N - t}

$$

Logic:

- The numerator $(s - m)$ is the number of items we *still need* to pick.
- The denominator $(N - t)$ is the number of items *remaining* in the file.
- This probability dynamically adjusts. If we haven't picked enough items recently, the probability increases. If we have picked many, it decreases.

Correctness: This guarantees that every item has an equal probability s/N of being selected.

I/O Efficiency:

- **Pros:** It is a single sequential scan.

- **Cons:** We must read the entire file (Cost $= N/B$). If $s \ll N$, this is inefficient. We would prefer an algorithm that skips blocks, but skipping requires random access, which is expensive on disk unless s is very small.

1.2 Algorithm B: Dictionary-based Sampling

If we need to sample m items where $m \ll N$, and we have random access:

1. Generate a random index $p \in [1, N]$.
2. Check if p is in a dictionary D (BST or Hash Table).
3. If not, insert p into D and pick item $S[p]$.
4. Repeat until $|D| = m$.

Complexity:

- Expect to try roughly m times (collisions are rare if $m \ll N$).
- Cost: $O(m \log m)$ insertions into Dictionary (BST).
- I/O: $O(m)$ random accesses.

---

2. Streaming Model & Reservoir Sampling (Unknown Length)

2.1 The Streaming Model

- **Constraint:** Data arrives as a sequence $x_1, x_2, \ldots$ and we see each item only once.
- **Unknown N:** We do not know when the stream ends.
- **Goal:** At any point in time t, we want to maintain a representative random sample of size s of all t items seen so far.

2.2 Reservoir Sampling Algorithm (Waterman's Algorithm)

We maintain a "Reservoir" (buffer) R of size s.

Algorithm:

1. **Initialization:** Put the first s items of the stream directly into R.
2. **Processing item t (where $t > s$):**

- Generate a random number $h \in [1, t]$.
- If $h \leq s$:
- **Swap:** Replace the item at index $R[h]$ with the new item x_t.
- Else ($h > s$):
- **Discard:** Ignore x_t.

Probability Logic: The probability of accepting the t-th item into the reservoir is: $$P(\text{keep } x_t) = \frac{s}{t}

$$

2.3 Proof of Uniformity

We must prove that after seeing t items, **every** item seen so far ($x_1 \ldots x_t$) has an equal probability $\frac{s}{t}$ of being in R.

Proof by Induction:

Base Case ($t = s$): Every item is in the reservoir with probability $1 = s/s$. Correct.

Inductive Step: Assume that after $t - 1$ items, every item x_i (where $i < t$) is in R with probability $\frac{s}{t-1}$. Now consider the arrival of item x_t.

1. **For the new item x_t:** By definition, it is selected with probability $\frac{s}{t}$. Correct.
2. **For an old item x_i ($i < t$) already in R:** x_i remains in R if it is **not** replaced by x_t.

x_i is removed only if:

- x_t is selected (Prob $= \frac{s}{t}$).
- **AND** the random index h chosen for replacement is exactly the index of x_i (Prob $= \frac{1}{s}$).

So, Prob(x_i removed) $= \frac{s}{t} \times \frac{1}{s} = \frac{1}{t}$.

Therefore, Prob(x_i survives) $= 1 - \frac{1}{t} = \frac{t-1}{t}$.

Total Probability:

$$P(x_i \in R \text{ at } t) = P(x_i \in R \text{ at } t-1) \times P(x_i \text{ survives})$$

Substitute the inductive hypothesis:

$$P(x_i \in R \text{ at } t) = \frac{s}{t-1} \times \frac{t-1}{t} = \frac{s}{t}$$

Conclusion: After step t, every item has probability s/t of being in the reservoir.

Intersection Algorithms

The "Intersection Problem" is a canonical operation in Search Engines (handling "AND" queries between inverted lists). **Problem:** Given two sorted lists L_1 (length n) and L_2 (length m), with $n \leq m$, return $L_1 \cap L_2$.

---

1. Basic Approaches

1.1 Merge-based Intersection

The standard approach is to place a pointer at the start of each list and advance the pointer pointing to the smaller value.

- **Algorithm:**
  - If $L_1[i] < L_2[j]$, advance i.
  - If $L_1[i] > L_2[j]$, advance j.
  - If $L_1[i] == L_2[j]$, add to result, advance both.
- **Complexity:** $O(n + m)$.
- **Critique:** This is optimal when $n \approx m$. However, if $n \ll m$ (e.g., searching for "Quicksort" [rare] AND "The" [common]), we waste time scanning the entirety of the long list L_2 ($O(m)$) just to verify n items.

1.2 Binary Search-based

We iterate through the short list L_1 and, for every element $x \in L_1$, perform a binary search in L_2.

- **Complexity:** $O(n \log m)$.
- **Comparison:** Better than Merge if $n \log m < m$.

---

2. Advanced Intersections

2.1 Doubling Search (Galloping)

This algorithm adapts to the distribution of matches. It avoids searching the entire remaining part of L_2 by estimating where the next match *could* be.

The Algorithm: Let i be the current position in L_2 (after finding the previous match). To find the next element $x \in L_1$ in L_2:

1. **Gallop:** Check indices $i+1, i+2, i+4, \ldots, i+2^k$ in L_2.
2. **Stop:** Stop when we find a window where $L_2[i+2^{k-1}] < x \leq L_2[i+2^k]$.
3. **Binary Search:** Perform standard binary search only within this specific window (size 2^{k-1}).

Complexity Analysis: Let d_j be the distance in L_2 between the $(j-1)$-th match and the j-th match. The cost to find the j-th element is $O(\log d_j)$. Total cost:

$$\text{Cost} = \sum_{j=1}^{n} O(\log d_j)$$

Since $\sum d_j \leq m$ (the total length of L_2), by Jensen's inequality (concavity of log), this sum is maximized when all d_j are equal ($d_j \approx m/n$).

$$\text{Total Cost} = O\left(n\left(1 + \log \frac{m}{n}\right)\right)$$

This is asymptotically optimal for comparison-based intersection.

2.2 Mutual Partitioning

A recursive, divide-and-conquer approach.

1. Pick the **median** of the shorter list L_1. Let this be p.
2. Binary search for p in L_2. This splits both lists into (Left, Right) sets.
3. **Recurse:** Solve Intersect($L_{1,left}, L_{2,left}$) and Intersect($L_{1,right}, L_{2,right}$).
4. *Swap roles:* In the recursive calls, if the "Left" part of L_2 becomes shorter than L_1's part, swap them so we always iterate/pivot on the shorter list.

- **Complexity:** $O(n(1 + \log(m/n)))$. Matches the Galloping bound.

2.3 Two-Level Memory Approach (Cache Blocking)

Standard algorithms ignore the cache. A simple scan might fetch a cache line (block L) but only use 1 item if the intersection is sparse. **Goal:** Perform intersection at the granularity of cache blocks.

Setup:

1. Partition arrays into blocks of size L (cache line size).
2. Create a meta-array A' containing the *first key* of every block in A. $|A'| = n/L$.

Algorithm:

1. **Filter:** Intersect B with the small meta-array A'. This identifies which blocks in A *might* contain elements from B.
2. **Refine:** Only load and intersect the specific blocks A_j that passed the filter.

Complexity:

$$\text{Cost} \approx O\left(\frac{n}{L} + m + m \cdot L\right)$$

- n/L: Scanning the meta-array.
- $m \cdot L$: Worst case, every item in B falls into a different block of A, forcing us to scan a full block of size L for each item.

---

3. Interpolation Search

Standard Binary Search blindly cuts the array in half ($mid = (low + high)/2$). **Interpolation Search** acts like humans searching a phonebook: if looking for "Zuck", we open near the end; if "Ada", near the beginning.

3.1 The Algorithm

We estimate the position of target x using the formula:

$$next = low + \lfloor \frac{x - A[low]}{A[high] - A[low]} \times (high - low) \rfloor$$

- **Logic:** It assumes a linear relation between the value of keys and their array indices.

3.2 Assumptions & Complexity

- **Requirement:** Keys must be drawn from a **Uniform Distribution**.
- **Performance:**
  - **Average Case:** $O(\log \log N)$.
  - **Worst Case:** $O(N)$ (if distribution is highly skewed, e.g., exponential).
- **Derivation:** The recurrence relation roughly follows $T(N) \approx T(\sqrt{N}) + O(1)$, which resolves to $\log \log N$.

3.3 Why it fails on Disk

Interpolation Search is terrible for external memory.

1. **Calculation:** The formula involves division/multiplication (CPU heavy, though negligible compared to I/O).
2. **Random Access:** The `next` position jumps unpredictably. It does not narrow down to a specific block B quickly enough to benefit from caching.
3. **Data Requirements:** Real-world data is rarely perfectly uniform.

   Conclusion: For in-memory uniform data, Interpolation Search is fast. For disk-based or skewed data, B-Trees or Galloping Search are superior.

Randomized Dictionaries

In this chapter, we explore dictionary data structures that rely on **randomization** to achieve balance, rather than complex deterministic rebalancing rules (like AVL or Red-Black trees).

---

1. Skip Lists

A **Skip List** is a probabilistic alternative to balanced trees. It consists of sorted linked lists arranged in levels.

1.1 Structure and Levels

- **Level L_0:** Contains all elements in sorted order.
- **Level L_i:** Contains a subset of elements from L_{i-1}.
- **Sentinels:** All levels start with $-\infty$ and end with $+\infty$.

Promotion Logic (The Coin Flip): When inserting an element x:

1. Insert x into L_0.
2. Flip a fair coin ($p = 0.5$).

- **Heads:** Promote x to level L_1. Flip again.
- **Tails:** Stop promoting.

3. Repeat until Tails occurs.

Biased Skip Lists Standard Skip Lists assume uniform access. If some keys are accessed more frequently, we want them higher up (shorter search path).

- **Idea:** Unlike the random coin flip, we use a **deterministic stack** based on access probability $p(x)$.
- **Height:** We force the height of key x to be proportional to $\log(1/p(x))$.
- **Result:** Frequent items stay near the top, behaving like a static optimal search tree.

1.2 Complexity Analysis

Space Analysis Let N be the number of elements.

- Prob(x exists at level i) $= 1/2^i$.
- Expected size of level i: $N/2^i$.
- Total Expected Space:

$$\sum_{i=0}^{\infty} \frac{N}{2^i} = N \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i = N \cdot 2 = O(N)$$

Search Time Analysis

- **Height (H):** The probability that a Skip List has height greater than $c \log N$ is extremely low ($1/N^{c-1}$). Thus, $H = O(\log N)$ w.h.p.
- **Search Path:** We start at the top-left. At any node:
  - If `next.key` $\leq$ target: Move **Right**.
  - Else: Move **Down**.
- **Expected Steps:** The number of steps is proportional to the height.

$$E[\text{Search Time}] = O(\log N)$$

1.3 I/O Analysis (The Weakness)

Skip Lists are **not** I/O efficient compared to B-Trees.

- **Pointer Chasing:** Moving "Right" or "Down" involves following a pointer to a potentially random memory location (or disk block).
- **No Blocking:** Unlike a B-Tree node, which aggregates B keys into one block, a Skip List node contains only one key and pointers.
- **I/O Cost:** In the worst case (cold cache), a search involves $O(\log N)$ I/Os.
  - Compare to B-Trees: $O(\log_B N)$.
  - Since $B \approx 1000$, $\log_B N$ is vastly smaller than $\log_2 N$.

---

2. Treaps (Tree + Heap)

A **Treap** is a binary tree that satisfies two properties simultaneously:

1. **BST Property (on Keys):** For any node u, keys in the left subtree are $< key(u)$, and keys in the right subtree are $> key(u)$.
2. **Heap Property (on Priorities):** For any node u, $priority(parent(u)) < priority(u)$ (Min-Heap variant).

2.1 Randomization & Balance

- **Priorities:** When a node is inserted, it is assigned a unique priority chosen uniformly at random.
- **Theorem:** A Treap with random priorities is isomorphic to a **Random Binary Search Tree (RBST)** (a BST constructed by inserting keys in a random permutation).
  - *implication:* The expected height is $O(\log N)$.

2.2 Rotations

Insertions are performed as standard BST insertions (placing the node as a leaf). This may violate the Heap property. **Fix:** Perform **Rotations** to bubble the node up until the Heap property is restored. Rotations preserve the BST ordering of keys.

2.3 Operations

Split(T, k) Divides Treap T into two Treaps: $T_{\leq}$ (keys $\leq k$) and $T_{>}$ (keys $> k$).

- **Algorithm:**
  1. Insert a "dummy" node with key k and priority $-\infty$.
  2. Because priority is minimal, it bubbles up to the **root** via rotations.
  3. The left child of the root is $T_{\leq}$, the right child is $T_{>}$.
  4. Remove the dummy node.
- **Complexity:** $O(\log N)$.

Merge(T_1, T_2) Joins two Treaps (assuming all keys in $T_1 <$ all keys in T_2).

- **Algorithm:**
  1. Create a dummy root with priority $-\infty$.
  2. Attach T_1 as left child, T_2 as right child.
  3. "Sink" the dummy root down (rotating with the child having higher priority) until it becomes a leaf.
  4. Delete the dummy leaf.
- **Complexity:** $O(\log N)$.

Delete(k) Deletion is the inverse of insertion.

1. Find node u with key k.
2. Set priority of u to $+\infty$ (for Min-Heap) or $-\infty$ (for Max-Heap).
3. **Rotate Down:** Swap u with its child having the higher/lower priority (depending on Heap type) to maintain Heap property locally.
4. Repeat until u is a leaf.
5. Cut u.

2.4 3-Sided Range Query

Query: Find all nodes (x, y) such that $q_1 \leq x \leq q_2$ and $y \leq q_3$ (Range on Key, Threshold on Priority).

Algorithm:

1. **Search Spines:** Find the paths to q_1 and q_2 in the tree.
2. **Identify Subtrees:** Identify the subtrees hanging "between" these two paths.
3. **Pruning:** For each candidate subtree:
   - Check the root's priority.
   - If $root.prio > q_3$: **Stop**. By the Heap property, no node in this subtree can satisfy the condition.
   - If $root.prio \leq q_3$: Report root and recurse.

- **Complexity:** $O(\log N + K)$ where K is the number of reported items.

---

3. Proof of Average Depth

We prove that the expected depth of a node in a Random BST (Treap) is $O(\log N)$.

Setup: Let keys be $x_1 < x_2 < \cdots < x_N$. Define indicator variable $A_{i,k}$:

$$A_{i,k} = \begin{cases} 1 & \text{if } x_i \text{ is an ancestor of } x_k \\ 0 & \text{otherwise} \end{cases}$$

Depth Formulation: The depth of node x_k is the number of its ancestors:

$$D_k = \sum_{i=1}^{N} A_{i,k}$$

By Linearity of Expectation:

$$E[D_k] = \sum_{i=1}^{N} P(x_i \text{ is ancestor of } x_k)$$

Ancestor Condition: In a Treap, x_i is an ancestor of x_k **if and only if** x_i has the **lowest priority** among all nodes in the range between them (inclusive). Let this range be $R_{ik} = \{\min(i,k), \ldots, \max(i,k)\}$.

- Size of range: $|k - i| + 1$.
- Since priorities are random, every node in R_{ik} is equally likely to be the minimum.

$$P(x_i \text{ is ancestor of } x_k) = \frac{1}{|k-i|+1}$$

Summation:

$$E[D_k] = \sum_{i=1}^{N} \frac{1}{|k-i|+1} = \underbrace{\sum_{j=1}^{k} \frac{1}{j}}_{\text{Left side}} + \underbrace{\sum_{j=1}^{N-k+1} \frac{1}{j}}_{\text{Right side}} - 1$$

These are **Harmonic Numbers** ($H_n \approx \ln n$).

$$E[D_k] \approx \ln k + \ln(N-k) \leq 2 \ln N \approx 1.39 \log_2 N$$

Conclusion: Expected depth is $O(\log N)$.

String Sorting and Tries

Sorting strings differs fundamentally from sorting atomic keys (like integers) because strings have variable lengths, and comparisons depend on prefixes.

---

1. String Sorting

1.1 The Challenge

Standard comparison-based sorting (MergeSort, HeapSort) has a lower bound of $\Omega(n \log n)$ **comparisons**. However, for strings, a single comparison is not $O(1)$. Comparing two strings s_1, s_2 takes $O(|LCP(s_1, s_2)|)$ time.

- **Total naive cost:** $O(L_{avg} \cdot n \log n) = O(N \log n)$, where N is the total number of characters.

- **Optimality:** This is suboptimal. We end up re-scanning long common prefixes repeatedly.
- **True Lower Bound:** $\Omega(d + n \log n)$, where d is the sum of lengths of **distinguishing prefixes** (the shortest prefix needed to distinguish each string from the others).

1.2 Multi-key Quicksort (3-way String Quicksort)

An elegant adaptation of Quicksort for variable-length strings.

Algorithm: Given a set R and current character index i (initially 0):

1. Pick a pivot string p. Let $c = p[i]$ be the pivot character.
2. **3-way Partition** R into:
   - $R_<$: Strings where $s[i] < c$.
   - $R_=$: Strings where $s[i] = c$.
   - $R_>$: Strings where $s[i] > c$.
3. **Recurse:**
   - `MKQS`$(R_<, i)$ (Pivot character was not matched, stay at index i).
   - `MKQS`$(R_>, i)$ (Pivot character was not matched, stay at index i).
   - `MKQS`$(R_=, i + 1)$ (**Crucial:** We matched character i, so advance to $i + 1$).

Complexity:

- **Time:** $O(d + n \log n)$. Matches the lower bound.
- **Why?** Characters in identifying prefixes (d) are participating in the "=" partition. Other comparisons contribute to the sorting cost ($n \log n$).

1.3 Radix Sorts

LSD (Least Significant Digit) Sorts from the last character to the first.

- **Requirement:** The inner sorting algorithm must be **Stable** (e.g., Counting Sort). If it weren't stable, sorting by character i would scramble the order established by character $i + 1$.
- **Complexity:** $O(L_{max} \cdot n)$.
- **Drawback:** Must touch every character, even if strings are distinct by the first letter.

MSD (Most Significant Digit) Sorts from the first character to the last (recursive buckets).

- **Variable Lengths:** Shorter strings are conceptually padded with a sentinel character (smaller than any alphabet char) to ensure they end up in the correct bucket.
- **Complexity:** $O(N)$ in practice, but overhead of recursion is high for small buckets.

---

2. Trie-based Structures

2.1 Standard Tries (Uncompacted)

A tree where edges are labeled with characters.

- **Node Structure:** An array of size σ (alphabet size).
- **Problem:** Space complexity is $O(N \cdot \sigma)$. Most nodes in a Trie have only 1 child (sparse), wasting huge amounts of memory for large alphabets (like Unicode).

2.2 Ternary Search Trees (TST)

Combines the time efficiency of Tries with the space efficiency of BSTs.

- **Node Structure:**
  - `char c` (Split Character)
  - `left` pointer ($< c$)

  - `eq` pointer ($= c$)
  - `right` pointer ($> c$)
- **Traversal:** Similar to Multi-key Quicksort. If matches `c`, follow `eq`. Else, follow `left/right`.
- **Space:** $O(N)$ nodes. No dependency on σ.

2.3 Compacted Tries (Patricia Tries)

Optimizes standard Tries by compressing paths of single-child nodes.

- **Compression:** A chain of nodes `a -> p -> p -> l -> e` is collapsed into a single edge labeled "apple".
- **Edge Label:** Stored as a triple $\langle \text{string_id}, \text{start}, \text{end} \rangle$.
- **Node Count:** Guaranteed $O(n)$ nodes (where n is number of strings), regardless of total length N. Every internal node has branching factor ≥ 2.

Patricia Trie Search: The 3 Phases Patricia Tries store only the first char of the edge label + the label length (to skip). This creates ambiguity. **Algorithm:**

1. **Blind Downward Traversal:** Follow edges matching the single character. Skip the number of characters indicated by the edge length. Do *not* check the skipped characters (blind).
2. **Leaf Selection:** We eventually hit a leaf (or fail). Let the leaf be string S.
3. **Upward Verification:** Calculate $LCP(P, S)$. If $LCP == |P|$, we found it. If not, the mismatch character determines the lexicographic relationship.

- *Why?* This avoids decoding the edge labels (pointers) during the traversal, reducing cache misses.

2.4 Array-based Solutions

- **Naive Array of Pointers:** `malloc` for every string. Causes massive cache misses (pointer chasing).
- **Giant Block:** Concatenate all strings into one large contiguous memory block (separated by `\0`). The "pointer" array becomes an array of integer offsets.
  - Improves locality significantly.
  - Allows pointer compression (offsets are smaller than 64-bit pointers).

2.5 Front Coding

Used for storing sorted dictionaries on disk (e.g., inside B-Tree leaves).

- **Idea:** Sorted strings share long prefixes.
- **Format:** Store shared prefix length ℓ + remaining suffix.
  - `alcatraz`
  - `alcohol` $\rightarrow$ `(3, ohol)` (matches 'alc')
  - `alcoholic` $\rightarrow$ `(7, ic)` (matches 'alcohol')

2.6 Two-Level Indexing Architecture

To handle massive dictionaries on disk, we combine **Front Coding** (disk) with **Tries** (memory). 1. **Disk Layout:** Sort strings and partition them into blocks of size B (e.g., 4KB). Inside each block, compress strings using **Front Coding**. 2. **Memory Layout:** Take the **first string** of every block (the "separator"). Build a **Compacted Trie** (or Patricia Trie) in RAM using only these separator strings. 3. **Search(P):** * Use the in-memory Trie to find the predecessor block B_i. *Load block B_i from disk (1 I/O).* Decompress/Scan the block to find P. * **Advantages:** High compression on disk, fast search (1 I/O), minimal RAM usage (only N/B keys stored in Trie).

Full Text Indexing

Full Text Indexing solves the problem of finding a pattern P in a text T efficiently. While Suffix Trees are powerful, they are space-heavy. **Suffix Arrays (SA)** combined with the **LCP Array** offer a space-efficient alternative that supports similar operations.

---

1. Suffix Array (SA)

1.1 Definition

Given a text T of length N, the Suffix Array `SA` is an array of integers from 0 to $N-1$. * **Content:** `SA[i]` is the starting position of the i-th lexicographically smallest suffix of T. * **Space:** N integers ($4N$ bytes), which is much smaller than a Suffix Tree pointer structure.

1.2 Binary Search on SA ($O(|P| \log N)$)

Since `SA` stores suffixes in sorted order, all suffixes starting with pattern P appear contiguously in `SA`. We can find the range $[L, R]$ of these suffixes using Binary Search.

Algorithm: 1. **Compare:** Compare P with the suffix starting at `SA[mid]`. 2. **Branch:** * If $P < T[\text{SA[mid]}]$, go Left. * If $P > T[\text{SA[mid]}]$, go Right. 3. **Repeat:** Standard binary search steps.

Complexity: * Number of steps: $O(\log N)$. * Cost per step: Comparison takes $O(|P|)$ in worst case. * **Total:** $O(|P| \log N)$.

1.3 LCP-LR Optimization ($O(|P| + \log N)$)

We can speed this up by avoiding redundant comparisons. We precompute two auxiliary arrays L_{lcp} and R_{lcp} for the binary search ranges.

- Let $l = LCP(P, \text{suffix}_{Low})$.
- Let $r = LCP(P, \text{suffix}_{High})$.
- When checking `mid`, we know the shared prefix between `Low` and `mid` (stored in L_{lcp}).
  - If $L_{lcp}[mid] > l$, then P matches `mid` up to l automatically, and the mismatch is further down.
  - If $L_{lcp}[mid] < l$, then P is determined by the mismatch character known from `Low`.
- **Result:** We only advance the character pointer on P (never backtrack). Total comparisons limited to $O(|P|)$.
- **Total:** $O(|P| + \log N)$.

1.4 Naive Construction Analysis

A naive approach to build the SA is using a standard sort (like `qsort`). * **Comparison Cost:** Comparing two suffixes takes $O(N)$ in the worst case (e.g., `aaaaa...`). * **Sort Cost:** $O(N \log N)$ comparisons. * **Total Complexity:** $O(N^2 \log N)$ on average, or $O(N^3)$ worst case. * **Critique:** This is unacceptably slow for large texts. Advanced algorithms (like DC3 or IS) can do this in $O(N)$, and Kasai computes LCP in $O(N)$.

---

2. LCP Array Construction (Kasai's Algorithm)

The **Longest Common Prefix (LCP)** array stores the length of the shared prefix between consecutive suffixes in the sorted SA.

$$LCP[i] = \text{length}(LCP(\text{suffix}_{SA[i]}, \text{suffix}_{SA[i-1]}))$$

2.1 The Algorithm (Linear Time)

Naive construction takes $O(N^2)$. Kasai's algorithm achieves $O(N)$ by iterating through suffixes in **text order** ($i = 0 \ldots N$), not SA order.

Comparison: Brute Force vs Kasai Example T = "banana$". * **Brute Force:** Compares every pair from scratch. Total comparisons can be quadratic. * **Kasai:** 1. Compute LCP for suffix `banana$` (pos 0). Result = 0. 2. Move to suffix `anana$` (pos 1). We know it shares a prefix with the predecessor of `banana$`. 3. If prev LCP was H, we start comparing from $H - 1$. We skip characters we know must match.

2.2 The Inequality Proof

Let $rank[i]$ be the position of suffix i in the Suffix Array. Let $H = LCP[rank[i]]$. We want to prove:

$$LCP[rank[i+1]] \geq H - 1$$

Proof: 1. Let suffix j be the immediate predecessor of suffix i in the SA. ($rank[j] = rank[i] - 1$). 2. They share a prefix of length H.

$$suff_i = c \cdot \alpha \ldots$$

$$suff_j = c \cdot \alpha \ldots$$

3. Now consider $suff_{i+1}$ and $suff_{j+1}$. They are obtained by dropping the first char c.

$$suff_{i+1} = \alpha \ldots$$

$$suff_{j+1} = \alpha \ldots$$

They clearly share a prefix of length $H - 1$. 4. $suff_{j+1}$ appears somewhere before $suff_{i+1}$ in the SA (lexicographical order is usually preserved unless the first char differed). 5. The LCP between $suff_{i+1}$ and its *immediate* predecessor in SA must be $\geq$ the LCP between $suff_{i+1}$ and *any* predecessor (including $suff_{j+1}$). 6. Therefore, the stored LCP value for $i + 1$ is at least $H - 1$.

2.3 Complexity Analysis

- **Increments:** In the `while` loop, we compare characters to increase H. Since H starts at 0 and max is N, total increments $\leq 2N$.
- **Decrements:** In every step $i \to i + 1$, H decreases by at most 1. Total decrements $\leq N$.
- **Total:** Time is proportional to increments + decrements = $O(N)$.

---

3. Suffix Trees from SA + LCP

A Suffix Tree can be viewed as the **Cartesian Tree** of the LCP array.

Construction: 1. Insert suffixes in SA order. 2. Use $LCP[i]$ to determine how high up the rightmost path of the tree we must go to attach the new leaf. * If $LCP[i]$ matches the string depth of a node u, attach as child. * If $LCP[i]$ falls in the middle of an edge, **Split** the edge and attach. 3. All operations are amortized $O(1)$ (similar to the "Rightmost path" logic in Cartesian tree building). 4. **Total Time:** $O(N)$.

---

4. Text Mining with SA

4.1 Repeated Substrings

Problem: Is there a substring of length l that appears r times? **Theorem:** Such a substring exists **iff** there is a contiguous block in the LCP array of size at least $r - 1$ where every value is $\geq l$. **Proof:** * If $LCP[k] \geq l$, then $SA[k]$ and $SA[k-1]$ share a prefix of length l. * If we have a block $LCP[i], \ldots, LCP[i + r - 2]$ all $\geq l$, then by transitivity (minimum of range), all suffixes $SA[i-1], \ldots, SA[i + r - 2]$ share a prefix of length l. * This set contains $(i + r - 2) - (i - 1) + 1 = r$ suffixes.

4.2 Distance Constraint (P, Q, k)

Problem: Given pattern P, pattern Q, and distance k. Find if there exists an occurrence of P and Q in T such that distance is $\leq k$.

Algorithm: 1. **Search:** Use Binary Search on SA to find the range of suffixes for P ($[sp, ep]$) and Q ($[sq, eq]$). 2. **Collect:** We have two sets of positions: $S_P = \{SA[i] \mid sp \leq i \leq ep\}$ and $S_Q = \{SA[j] \mid sq \leq j \leq eq\}$. 3. **Merge/Plane Sweep:** * Sort S_P and S_Q (if not already sorted by position). * Iterate through the sorted lists. For each $p \in S_P$, check if any $q \in S_Q$ falls in $[p - k, p + k]$. * Since lists are sorted, this check is linear $O(|S_P| + |S_Q|)$.

Complexity: $O(N)$ (Build SA) + $O(|P| \log N + |Q| \log N)$ (Search) + $O(\text{occ} \log \text{occ})$ (Sort positions) + $O(\text{occ})$ (Sweep).

Hashing Protocols

This chapter covers advanced hashing techniques that provide probabilistic guarantees on performance, moving beyond simple heuristics.

---

1. Universal Hashing

Standard hashing uses a fixed function $h(x)$. This is vulnerable to adversarial attacks (a user can deliberately input keys that all map to the same slot, causing $O(N)$ performance). **Universal Hashing** solves this by selecting a hash function *randomly* from a carefully designed family at runtime.

1.1 The Definition

A family of functions $\mathcal{H} = \{h : U \to \{0, \ldots, m-1\}\}$ is **Universal** if, for any two distinct keys $x, y \in U$:

$$P_{h \in \mathcal{H}}(h(x) = h(y)) \leq \frac{1}{m}$$

- **Implication:** The collision probability is the same as if the hash function were truly random (uniform).
- **Chain Length:** If we use chaining with a universal hash function, the expected length of a chain is $1 + \alpha$ (where $\alpha = N/m$), guaranteeing $O(1)$ average access.

1.2 Example: The Class $\mathcal{H}_{a,b}$

We can construct a universal family using a large prime $p > |U|$. For any $a \in \{1, \ldots, p-1\}$ and $b \in \{0, \ldots, p-1\}$, define:

$$h_{a,b}(x) = ((ax + b) \mod p) \mod m$$

- **Randomization:** At program start, we pick random integers a and b.
- **Why it works:** The map $x \to (ax + b) \mod p$ is a bijection on the field $\mathbb{Z}_p$. It "scrambles" the input uniformly before mapping to m buckets.

---

2. The Power of Two Choices

In standard chaining, even with a perfect hash function, the maximum chain length (Max Load) grows with N.

- **One Choice:** If we throw N balls into N bins randomly:

$$\text{Max Load} \approx \frac{\ln N}{\ln \ln N}$$

- **Two Choices:** If for every ball, we compute **two** hashes $h_1(x), h_2(x)$, inspect the load of both buckets, and place the ball in the **less loaded** one:

$$\text{Max Load} \approx \frac{\ln \ln N}{\ln 2} + O(1)$$

- **Impact:** This is an exponential improvement. For $N = 10^6$, the max load drops from ≈ 9 to ≈ 3.

---

3. Cuckoo Hashing

Cuckoo Hashing leverages the "Power of Two Choices" to achieve $O(1)$ **Worst-Case Lookup Time**.

3.1 The Algorithm

- **Structure:** Two tables T_1, T_2 of size m. Two universal hash functions h_1, h_2.
- **Invariant:** A key x is stored **either** at $T_1[h_1(x)]$ **or** at $T_2[h_2(x)]$.

Lookup(x)

1. Check $T_1[h_1(x)]$. If found, return.
2. Check $T_2[h_2(x)]$. If found, return.
3. Else, not found.

- **Cost:** Exactly 2 memory accesses. **Deterministic** $O(1)$.

Insert(x)

1. Try to place x in $T_1[h_1(x)]$.
2. If empty, done.
3. If occupied by key y, **kick out** y and replace with x.
4. Now insert y into its *alternative* location (e.g., if y was in T_1, try $T_2[h_2(y)]$).
5. If that spot is occupied by z, kick out z and repeat.
6. **Rehash:** If this process loops too long ($> MaxLoop$), assume a cycle and rebuild tables with new hash functions.

3.2 The Cuckoo Graph

We can model the state as a graph:

- **Nodes:** The $2m$ slots in the tables.
- **Edges:** For every key k, an edge connects $h_1(k) \leftrightarrow h_2(k)$.
- **Insertion:** Insertion succeeds if the connected component containing the new edge is a **Tree** or has at most **One Cycle** (Unicyclic).
  - We can orient edges to point to where the key *is currently stored*.
  - A tree or 1-cycle component can always be oriented such that every key has a home.
- **Failure:** If the component contains **two cycles** (a "Bicycle" or "Figure-8"), insertion is impossible. The keys will kick each other around infinitely.

3.3 Analysis

- **Load Factor:** Cuckoo hashing works if $\alpha < 0.5$ (tables less than half full).
- **Rehash Probability:** With $\alpha < 0.5$, the probability of forming a bicycle is $O(1/N)$.
- **Amortized Cost:** Since rehashing is rare, expected insertion time is **constant** $O(1)$.

3.4 Proof of Path/Cycle Probability

We want to bound the probability of a long sequence of evictions (a long path in the graph). **Theorem:** For any two positions i, j, if $m \geq 2cn$ (load $< 1/2c$), then:

$$P(\text{path } i \to \cdots \to j \text{ of length } L) \leq \frac{1}{m \cdot c^L}$$

Proof by Induction on L: 1. **Base ($L = 1$):** Path of length 1 means an edge exists between i and j.

$$P(\exists k : h_1(k) = i \wedge h_2(k) = j) \leq \sum_k \frac{1}{m^2} = \frac{n}{m^2} \leq \frac{1}{2cm}$$

2. **Step:** A path of length L implies a path of length $L - 1$ to some node z, and an edge from z to j.

$$P(\text{path } L) \leq \sum_z P(\text{path } i \to z \text{ len } L - 1) \cdot P(\text{edge } z \to j)$$

$$\leq m \cdot \left(\frac{1}{mc^{L-1}}\right) \cdot \frac{1}{cm} = \frac{1}{mc^L} 3.$$

Conclusion: The probability of a path decreases exponentially with length. The probability of a cycle (path from i to i) is very low, and a double cycle ("Bicycle") is $O(1/N^2)$.

Filters and Perfect Hashing

This chapter focuses on space-efficient structures for set membership and static dictionary problems.

---

1. Minimal Ordered Perfect Hashing (MOPHF)

1.1 The Goal

Given a **static** set of keys S ($|S| = N$), we want to construct a hash function $h(x)$ such that:

1. **Perfect:** No collisions for $x \in S$.
2. **Minimal:** The range of $h(x)$ is exactly $\{1, \ldots, N\}$.
3. **Ordered:** If $x < y$, then $h(x) < h(y)$. This means $h(x)$ returns the **rank** of x in S.

1.2 Construction (3-Hypergraph / Bucketing)

We use a 2-level construction involving random hypergraphs.

- **Hash Functions:** We pick two universal hash functions h_1, h_2 mapping to a larger range $M' \approx cN$.
- **The Graph:** Construct a graph where nodes are $0 \ldots M' - 1$ and each key $k \in S$ is an edge between $h_1(k)$ and $h_2(k)$.
- **Acyclicity:** We retry choosing h_1, h_2 until this graph is **acyclic**.
- **Labeling (g):** We assign values to nodes (g array) such that for every edge $k = (u, v)$, the rank of k is derived from $g[u] + g[v]$.
$$h(k) = (g(h_1(k)) + g(h_2(k))) \mod N$$
- **Efficiency:**
  - **Time:** $O(N)$ expected construction, $O(1)$ worst-case query.
  - **Space:** $O(N)$ bits (storing the g array).

---

2. Bloom Filters

A probabilistic data structure that tests membership.

- **False Negatives:** Impossible. (If it says "No", it's definitely not there).
- **False Positives:** Possible with probability f. (If it says "Yes", it might be there).

2.1 Algorithm

- **Storage:** An array B of M bits, initialized to 0.
- **Hashes:** k independent hash functions $h_1, \ldots, h_k$.

Insert(x): Set $B[h_i(x)] = 1$ for all $i = 1 \ldots k$.

Query(x): Return **True** if $B[h_i(x)] == 1$ for all $i = 1 \ldots k$. Else return **False**.

2.2 Error Analysis

Let N be the number of inserted items.

1. **Prob(Bit is 0):** The probability that a specific bit is *not* set by one specific hash function during one insertion is $1 - 1/M$. After N insertions (k hashes each):

$$p_0 = \left(1 - \frac{1}{M}\right)^{kN} \approx e^{-kN/M}$$

2. **False Positive Rate (f):** A query for a non-existent element returns true if all k corresponding bits happen to be 1.

$$f = (1 - p_0)^k \approx \left(1 - e^{-kN/M}\right)^k$$

2.3 Optimal k

To minimize f, we differentiate with respect to k. The optimal number of hash functions is:

$$k = \frac{M}{N} \ln 2 \approx 0.7 \frac{M}{N}$$

Substituting this back into the error formula, the optimal error rate is:

$$f_{opt} = \left(\frac{1}{2}\right)^k = (0.6185)^{M/N}$$

2.4 Application: Approximate Set Intersection

Scenario: Two servers A and B want to compute $|A \cap B|$ (e.g., Netflix CDNs syncing files) without sending the full dataset. **Protocol:**

1. A sends $BF(A)$ to B.
2. B checks every item $y \in B$ against $BF(A)$.
3. B counts the matches. **Error:** The result is $|A \cap B| +$ False Positives.

- Expected False Positives $\approx |B \setminus A| \times f$.
- To improve accuracy, B can send the "suspected" intersection back to A for exact verification (2-round protocol).

---

3. Spectral Bloom Filters (SBF)

Standard Bloom Filters cannot handle **deletions** or **frequency estimation**.

3.1 Structure

Replace the bit array B with an array of **counters** C.

- **Insert(x):** Increment $C[h_i(x)]$ for all i.
- **Delete(x):** Decrement $C[h_i(x)]$.
- **Query Frequency(x):**
$$\text{freq}(x) \approx \min_i \{C[h_i(x)]\}$$
  - *Why Min?* Collisions only *add* to the counter. The counter with the least noise is the closest upper bound to the true frequency.

Data Compression

We focus on **lossless** statistical compression. The goal is to represent a message S of length n using the minimum number of bits possible, bounded by the entropy of the source.

1.1 Compression Models

1. **Static:** The frequency model is fixed (e.g., standard English letter frequencies). Fast, but poor compression if data deviates from standard.
2. **Semi-Dynamic:** Two-pass. Pass 1 counts frequencies and builds the model (stored in header). Pass 2 encodes data. Good compression, slower.
3. **Dynamic:** One-pass. The model is updated on the fly as symbols are read. Decoder mirrors the updates.

---

2. Information Theory Basics

2.1 Entropy (H_0)

Entropy is a measure of the information content or "randomness" of a source. For a source alphabet Σ where each symbol σ appears with probability $P(\sigma)$, the **0-th order Entropy** is:

$$H_0 = \sum_{\sigma \in \Sigma} P(\sigma) \log_2 \left(\frac{1}{P(\sigma)} \right) = -\sum_{\sigma \in \Sigma} P(\sigma) \log_2 P(\sigma)$$

* **Meaning:** H_0 is the theoretical lower bound on the average number of bits per symbol needed to represent the source. No compressor can beat H_0 (on average) if it treats symbols independently.

2.2 Kraft's Inequality

For any instantaneous (prefix-free) code with codeword lengths $\ell_1, \ell_2, \ldots, \ell_{|\Sigma|}$, the lengths must satisfy:

$$\sum_{i=1}^{|\Sigma|} 2^{-\ell_i} \leq 1$$

* **Why?** This condition ensures that the codewords can be mapped to the leaves of a binary tree, which guarantees unique decodability without lookahead.

---

3. Huffman Coding

Huffman coding is a greedy algorithm that produces an optimal prefix-free code for a given distribution.

3.1 Construction (Min-Heap)

1. **Init:** Create a leaf node for each symbol σ with weight $P(\sigma)$. Insert all into a Min-Heap.
2. **Loop:** While Heap size > 1:
   - Extract two nodes with smallest weights: u, v.
   - Create a new internal node *parent* with weight $w(u) + w(v)$.
   - Make u and v children of *parent*.
   - Insert *parent* back into Heap.
3. **Result:** The code for symbol σ is the path from Root to Leaf σ (0 for left, 1 for right).

3.2 Canonical Huffman

Standard Huffman trees are hard to store (need pointers). **Canonical Huffman** allows us to reconstruct the code using **only** the lengths of the codewords.

Reconstruction Algorithm: 1. **Input:** Array `num[L]` (count of symbols with length L) and `symb[L]` (list of symbols of length L, sorted lexicographically). 2. **Compute `fc` (First Codeword):** We calculate the integer value of the first codeword for each length L. * Start from max length: $fc[max_len] = 0$. * Iterate backwards from $i = max - 1$ down to 1:

$$fc[i] = \lfloor \frac{fc[i+1] + num[i+1]}{2} \rfloor$$

* *Logic:* This formula effectively performs a right shift. We take the "next available" integer at level $i+1$ and shift it right to find the prefix at level i. 3. **Assign:** For length L, the k-th symbol in `symb[L]` gets code $fc[L] + k$.

3.3 Extended Huffman

Standard Huffman has an overhead of +1 bit per symbol in the worst case ($H \leq L < H + 1$). * **Idea:** Group symbols into blocks of size k (k-tuples). * **Alphabet:** New alphabet size $|\Sigma|^k$. * **Overhead:** The +1 bit overhead is distributed over k symbols.

$$L_{avg} \approx H + \frac{1}{k}$$

* **Trade-off:** The tree size grows exponentially with k.

---

4. Arithmetic Coding

Huffman has a limitation: it must use an integer number of bits (at least 1) per symbol. **Arithmetic Coding** overcomes this by mapping the entire message to a single number (interval).

4.1 The Interval Concept

We represent the current state as an interval $[L, L + S)$ within $[0, 1)$. * Initially: $[0, 1)$. * Each symbol σ owns a sub-interval proportional to $P(\sigma)$. * When we process σ, we "zoom in" to its sub-interval.

4.2 Encoding Algorithm

Let $f(\sigma)$ be the cumulative probability (sum of $P(x)$ for all $x < \sigma$). For each symbol c in message: 1. New Size: $S_{new} = S_{old} \times P(c)$ 2. New Start: $L_{new} = L_{old} + S_{old} \times f(c)$ 3. Update $S \leftarrow S_{new}, L \leftarrow L_{new}$.

Result: A number $V \in [L_{final}, L_{final} + S_{final})$.

4.3 Truncation and Correctness

We output $d = \lceil \log_2(2/S_{final}) \rceil$ bits. Ideally, we send the midpoint $x = L + S/2$. However, binary truncation produces $\hat{x}$. **Theorem:** The truncation error $x - \hat{x} \leq 2^{-d} \leq S/2$. This ensures the transmitted value $\hat{x}$ remains within the final interval $[L, L + S)$, guaranteeing correct decoding.

4.4 Comparison: Huffman vs. Arithmetic

Consider a symbol with $P(A) = 0.99$. * **Entropy:** $H(A) = \log_2(1/0.99) \approx 0.014$ bits. * **Huffman:** Must assign at least 1 bit to 'A'. Efficiency is terrible (1 vs 0.014). * **Arithmetic:** The interval shrinks by factor 0.99. After 100 'A's, size is $0.99^{100} \approx 0.36$. We still barely need 1-2 bits to encode 100 symbols. * **Theorem:** Arithmetic coding uses at most $nH_0 + 2$ bits total. The overhead is negligible for large n.

---

5. Dictionary-Based Compression (LZ77)

Unlike statistical compressors (Huffman/Arithmetic) which use 0-th order entropy, **Lempel-Ziv (LZ77)** exploits repeating substrings ("phrases") to achieve higher-order compression.

5.1 The Algorithm

It parses the text T into a sequence of triples $\langle d, l, c \rangle$. * d **(Distance):** How far back to look for the copy. * l **(Length):** Length of the match. * c **(Next Char):** The literal character immediately following the match.

Encoding Example: $T =$ aacaacabc... 1. Start: No history. Emit $\langle 0, 0, \text{'a'} \rangle$. 2. Next 'a': Match found at dist 1? Emit $\langle 1, 1, \text{'c'} \rangle$. 3. Next 'aac': Longest match is "aac" (seen at start). Emit $\langle 3, 3, \text{'a'} \rangle$.

Decoding: Use a circular buffer (Window).

```
for (triple <d, l, c> : input) {
    start = cursor - d;
    for (i = 0; i < l; i++)
        out[cursor + i] = out[start + i]; // Copy byte by byte
    cursor += l;
    out[cursor++] = c;
}
```

Note: If $l > d$, the copy overlaps (e.g., "aaaaa" copying "a"). The byte-by-byte loop handles this naturally (run-length encoding).

5.2 Sliding Window & Hash Table

- **Sliding Window:** To bound search time and memory, we only look back W bytes (e.g., 32KB in Gzip, MBs in modern tools like Brotli).
- **Optimization:** Use a **Hash Table** indexed by **3-grams** (3 char sequences) to quickly find match candidates.
  - Key: `T[i]T[i+1]T[i+2]`
  - Value: List of positions where this trigram occurred.
  - Search: Check candidates, perform brute-force extension to find longest match.

5.3 LZSS Variant

LZ77 always emits a triple, even if no match is found (overhead). **LZSS** uses a 1-bit flag to distinguish:

- **0 + char:** Literal (uncompressed).
- 1 + $\langle d, l \rangle$: Copy reference.

  This removes the "next char" redundancy and improves compression for random data.

---

6. Integer Coding

LZ77 produces a stream of integers (distances and lengths). Storing them as fixed 32-bit integers is wasteful because small values are much more frequent.

6.1 Unary Coding

Represent x as $x - 1$ ones followed by a zero (or vice versa).

- $1 \to 10$
- $2 \to 110$
- $3 \to 1110$
- **Optimality:** Optimal for **Exponential Distributions** ($P(x) \approx 2^{-x}$).
- **Bad for:** Large numbers (x bits).

6.2 Elias Gamma Coding (γ)

Used for distributions that follow a Power Law ($P(x) \approx 1/x^2$).

Structure: $\underbrace{00\ldots0}_{N} \cdot \text{bin}(x)$

Where $N = \lfloor \log_2 x \rfloor$ (length of binary x minus 1).

- $13 \to \text{bin}(13) = 1101$ (len 4). Prefix with 3 zeros. $\gamma(13) = 0001101$.
- **Length:** $2\lfloor \log_2 x \rfloor + 1$ bits.

6.3 Elias Delta Coding (δ)

Better for larger integers. Gamma encodes the length of the number, followed by the number.

Structure: $\gamma(\text{len}(\text{bin}(x))) \cdot \text{bin}(x)_{\text{suffix}}$

- Slower to decode but asymptotically shorter for very large x.

6.4 Variable Byte Coding (VByte)

Bit-level codes (γ, δ) are slow due to unaligned memory access. VByte is byte-aligned.

- **Format:** Split x into 7-bit chunks.
- **Flag:** The 8th bit is `1` if more bytes follow, `0` if this is the last byte.
- **Pros:** Very fast decoding.
- **Cons:** Compression ratio worse than bit-level codes for small numbers.

6.5 Rice Coding (Golomb Family)

Rice coding is a specialized version of Golomb coding where the divisor is a power of 2 ($M = 2^k$). It is highly efficient for distributions where small numbers are frequent but large outliers exist (e.g., geometric distributions) and allows for very fast bitwise operations1111.

Given an integer $x > 0$ and a fixed parameter k:

1. **Quotient:** $q = \lfloor (x-1)/2^k \rfloor$ (implemented as right shift `(x-1) >> k`)2.
2. **Remainder:** $r = x - 1 - (q \cdot 2^k)$ (implemented as bitwise mask `(x-1) & ((1<<k) - 1)`)3.
3. **Format:** The code consists of $q + 1$ in Unary followed by r in Binary (using exactly k bits) 4.

- **Example: Encoding** 20 **with** $k = 4$ **(**$R_4(20)$**):**
  - $q = \lfloor 19/16 \rfloor = 1$.
  - $r = 19 \pmod{16} = 3$.
  - Unary part $(q + 1)$: Encode 2 as `10` (or `01`).
  - Binary part (r): Encode 3 in 4 bits as `0011`.
  - **Result:** `1000115`.
- **Random Access:** Unlike Gamma/Delta, Rice coding supports random access on the compressed stream. By using Rank/Select data structures on the unary parts (escaped symbols), one can jump to the i-th encoded integer without fully decoding the stream6.

6.6 PForDelta (Patched Frame of Reference)

Designed for very fast decompression (SIMD-friendly) in search engines. It optimizes for the case where the vast majority (e.g., 90%) of values fit within a small range $[base, base + 2^b - 2]$.

The Algorithm:

1. **Frame of Reference:** Determine a *base* value and a bit-width b.
2. **Encoding:** For a number x, if $0 \leq x - base < 2^b - 1$, store $(x - base)$ using b bits8.
3. **Exceptions (Patched):** If x is an outlier (too large):
   - Store an **Escape Symbol** (all 1s, i.e., $2^b - 1$) in the b-bit slot9.
   - Store the actual value of x in a separate **Exception Array** (typically using full 32-bit integers)10.

- **Space Complexity:** $N \cdot b + N_{exc} \cdot 32$ bits. The goal is to choose b such that N_{exc} is small (~10%).
- **Example:** Range $[0, 6]$ ($b = 3$, escape is `111`).
  - Input: `1, 0, 9`.
  - `1` $\rightarrow$ `001`.
  - `0` $\rightarrow$ `000`.
  - `9` (Outlier) $\rightarrow$ Write `111` to stream, write `9` to Exception Array 12.

6.7 Interpolative Coding

A recursive coding scheme for **strictly increasing sequences** (e.g., document IDs in inverted lists). It significantly outperforms Huffman and Entropy coders for clustered data by exploiting the constraints of the sequence13.

Problem: Encode a subsequence $S[l \ldots r]$ given knowledge that all values satisfy $low \leq S[i] \leq high$ 14.

Algorithm:

1. **Pick Middle:** Let $m = \lfloor (l + r)/2 \rfloor$. We must encode $S[m]$.
2. **Tighten Bounds:** Since the sequence is strictly increasing ($S[i] < S[i + 1]$), $S[m]$ is constrained by its neighbors:
   - It must be at least $low + (m - l)$ (leaving room for $m - l$ distinct items before it).
   - It must be at most $high - (r - m)$ (leaving room for $r - m$ distinct items after it).
   - Let range $R = [low + m - l,\ \ high - r + m]$ 16.
3. **Encode:** Store the offset of $S[m]$ within range R using standard binary encoding. Length $= \lceil \log_2(|R|) \rceil$.

4. **Recurse:**
   - **Left:** Encode $S[l \dots m-1]$ with bounds $[low, S[m]-1]$.
   - **Right:** Encode $S[m+1 \dots r]$ with bounds $[S[m]+1, high]$.

- **Implicit Encoding (Zero bits):** If the sequence is dense (e.g., $10, 11, 12$), the calculated range R for the middle element will have size 1. The log of 1 is 0. The algorithm emits **0 bits**, implicitly encoding the numbers purely by their constraints.

6.8 Elias-Fano Encoding

A specialized encoding for **monotone increasing sequences** (sorted integers) that allows **constant-time access** ($O(1)$) to the i-th element, making it extremely powerful for search engines and inverted indexes.

Algorithm:

Given a sorted sequence $S = s_1, \dots, s_n$ with max value $u = s_n + 1$:

1. **Parameters:**
   - Low-bits length: $\ell = \lfloor \log_2(u/n) \rfloor$.
   - High-bits length: $h = \lceil \log_2 u \rceil - \ell$.
2. **Decomposition:** Each number s_i is split into a **Low Part** (last ℓ bits) and a **High Part** (remaining upper bits).
   - $L[i] = s_i \pmod{2^\ell}$.
   - $H[i] = \lfloor s_i / 2^\ell \rfloor$.
3. **Storage:**
   - **Array L:** Stores the low bits of all numbers explicitly ($n \times \ell$ bits).
   - **Array H:** Stores the high parts using a **Unary-like** encoding. We construct a bitvector where we write $k$zeros followed by a 1, where k is the number of items having the specific high value j.
   - Conceptually: It is a bitvector of length $2n + n\lceil \log(u/n) \rceil$ bits.

Complexity & Space:

- **Space:** $2n + n \log(u/n)$ bits. This is very close to the information-theoretic lower bound (entropy of the gap distribution).
- **Access(i):** To retrieve $S[i]$, we combine:
  - Low part: Read ℓ bits from L at offset $i \cdot \ell$.
  - High part: Use **Select_1(i)** on the H array to find the position of the i-th 1. The number of zeros before it determines the high value.
  - $S[i] = \text{High} \cdot 2^\ell + \text{Low}$.

Operations:

- **Access(i):** Returns $S[i]$ in $O(1)$ time.
- **NextGEQ(x):** Returns the smallest $S[i] \geq x$. Uses `Select_0` on the high part to jump directly to the bucket where x might be, avoiding full decompression.

Rank and Select

Succinct data structures aim to store data using space close to the information-theoretic lower bound while supporting operations efficiently. The fundamental building blocks are **Rank** and **Select** on bitvectors.

1. Problem Definition

Given a binary array $B[1 \dots n]$ of length n. * $\mathbf{Rank}_b(i)$: Returns the number of bits equal to b in the prefix $B[1 \dots i]$. * Note: $\text{Rank}_0(i) = i - \text{Rank}_1(i)$. We only need to implement Rank_1. * $\mathbf{Select}_b(j)$: Returns the position of the j-th occurrence of bit b.

1.1 Naive Solutions

Let m be the number of 1s in B. 1. **Array of Positions:** Store indices of all 1s. * Space: $m \log n$ bits. * Select: $O(1)$ (Array access). * Rank: $O(\log m)$ (Binary Search). 2. **Prefix Sums:** Store cumulative count for every position. * Space: $n \log n$ bits. (Huge! $32n$ or $64n$). * Rank: $O(1)$. * Select: $O(\log n)$ (Binary Search).

---

2. Succinct Solution ($n + o(n)$ bits)

We keep the original bitvector B (n bits) and add auxiliary structures taking sub-linear space $o(n)$ to support $O(1)$ operations.

2.1 Two-Level Directory (Jacobson's Structure)

We divide B into blocks.

1. **Superblocks (Size Z):**
   - Store **Absolute Rank**: The total number of 1s before the start of the superblock.
   - Size: $Z = (\log n)^2$.
   - Space: $(n/Z) \times \log n = n/\log n$ bits ($o(n)$).
2. **Blocks (Size z):**
   - Store **Relative Rank**: The number of 1s from the start of the superblock to the start of this block.
   - Size: $z = \frac{1}{2} \log n$.
   - Space: $(n/z) \times \log Z = (2n/\log n) \times \log((\log n)^2) = O(n \frac{\log \log n}{\log n})$ bits ($o(n)$).
3. **In-Block Query:**
   - To solve Rank within a small block z, we scan.
   - Since z is tiny (e.g., 32 or 64 bits), it fits in a CPU register.
   - **Modern CPU:** Use `POPCOUNT` instruction to count 1s in $O(1)$.
   - **Theoretical:** Use the **Four Russians Trick**. Precompute a table T of all 2^z bit patterns.
     - $T[\text{pattern}][\text{offset}]$ stores the rank.
     - Table size is negligible ($2^{\log n/2} = \sqrt{n}$).

2.2 Query Algorithm

To compute $\text{Rank}_1(i)$: 1. **Superblock Index:** $S = \lfloor i/Z \rfloor$. Fetch `Super[S]`. 2. **Block Index:** $b = \lfloor i/z \rfloor$. Fetch `Block[b]`. 3. **In-Block:** Scan bits from start of block b to offset $i \pmod z$.

$$\text{Rank}(i) = \text{Super}[S] + \text{Block}[b] + \text{PopCount}(B[\dots])$$

* **Total Time:** $O(1)$. * **Total Space:** $n + o(n)$ bits.

2.3 Example Trace

$B = [010111010101110001]$. $n = 18$. * **Superblocks ($Z = 6$):** * SB_0 (Pos 0): 0. * SB_1 (Pos 6): $\text{pop}(010111) = 4$. * SB_2 (Pos 12): $4 + \text{pop}(010101) = 4 + 3 = 7$. * SB_3: $7 + 3 = 10$. * **Blocks ($z = 3$):** * b_0 (in SB_0): 0. * b_1 (in SB_0): $\text{pop}(010) = 1$. * ... * **Query Rank$_1(11)$:** * Index 11 (0-based? Let's assume 1-based 11th bit). * Superblock: $\lfloor 11/6 \rfloor = 1$ (Indices 0,1). Use $SB_1 = 4$. * Block in SB_1: Offset $11 - 6 = 5$. $\lfloor 5/3 \rfloor = 1$. Use $b_1 = 1$ (Relative to SB). * In-Block: Offset 5 (mod 3) = 2. Bits `010` (from pos 9,10,11). * Scan first 2 bits of `010`? No, offset 2 means 3rd bit. * Let's assume bits at 9,10,11 are `0,1,0`? No, B is `010101...` * Bits in block: `101`? * **Total:** $4 + 1 + 1 = 6$.

---

3. Compressed Solution (Elias-Fano)

If the bitvector is sparse (few 1s), we can do better than n bits. We store the **positions of the 1s** as an increasing sequence S using **Elias-Fano**.

3.1 Operations

- **Select$_1(i)$**: This is exactly **Access(i)** on the EF-encoded sequence.
  - Returns the position of the i-th 1.
  - **Time:** $O(1)$.
- **Rank$_1(j)$**: This is finding the index i such that $S[i] \leq j < S[i+1]$.
  - Equivalent to **NextGEQ($j+1$)**. The result x is the value $\geq j+1$.
  - The rank is the index of the value before it.
  - **Time:** $O(\log(u/m))$.

3.2 Recursive Definition

Elias-Fano itself relies on a bitvector H (unary high parts). To allow $O(1)$ operations on EF, we need Rank/Select on H. * Since H is dense (or we just need succinctness), we use the **Succinct ($n + o(n)$)** structure for H. * **Stack:** Compressed Problem $\rightarrow$ Elias-Fano $\rightarrow$ Succinct Bitvector on H.

Exam Questions

This document collects past exam questions, transcribed from handwritten notes and PDF archives.

Exam: 23 June 2025

Q1: List Intersection

Problem: Compute the intersection between the two lists:

$$L_1 = (1, 2, 3, 8, 9, 15, 20, 35, 40)$$

$$L_2 = (2, 5, 35)$$

Perform the intersection using: 1. **Mutual Partitioning** algorithm. 2. **Two-level approach** with block size $b = 3$ (for the longer list). Sol 23/06/25 Q1|See Solution

Q2: Elias-Fano Operations

Problem: Given the following Elias-Fano encoding of a sequence S: * $L = 01\ 11\ 00\ 01\ 01\ 00\ 11\ 11\ 00\ 11\ 00$ * $H = 110\ 110\ 10\ 0\ 10\ 10\ 10\ 110\ 0\ 0\ 10\ 0\ 0\ 0\ 0\ 0$

1. **Decompress** all integers encoded.
2. Show the execution of **Access(8)**.
3. Show the execution of **NextGEQ(31)**. Sol 23/06/25 Q2|See Solution

Q3: Rank Data Structure

Problem: Given the binary array:

$$B[1, 24] = 01\ 11\ 10\ 01\ 01\ 01\ 11\ 11\ 00\ 11\ 00\ 11$$

1. Build the data structure to solve the **Rank** operation in $O(1)$ time. * Superblock size $Z = 6$. * Block size $z = 2$. 2. Show how to solve **Rank(15)** using this structure. *(Hint: 15 bits are set to 1 in B).* Sol 23/06/25 Q3|See Solution

Q4: Succinct Tree Algorithms

Problem: A binary tree of n nodes is encoded succinctly in a binary array $B[1, 2n + 1]$ (likely LOUDS or BP) and an array $L[1, n]$ of node labels. **Task:** Design an algorithm that counts the number of nodes labeled "A" that have a **grandparent** labeled "B". Sol 23/06/25 Q4|See Solution

Solutions

Solutions: 23 June 2025

Sol 23/06/25 Q1

1. Mutual Partitioning: * Pivot selection: Median of shorter list L_2 is 5. * Search 5 in L_1: Found between 3 and 8. * **Split:** * $L_{1,left} = (1, 2, 3)$, $L_{1,right} = (8, 9, 15, 20, 35, 40)$. * $L_{2,left} = (2)$, $L_{2,right} = (35)$. * **Recurse Left:** $(1, 2, 3) \cap (2)$. Pivot 2. Match! **Output: 2**. * **Recurse Right:** $(8 \ldots 40) \cap (35)$. Pivot 35. Match! **Output: 35**.

2. Two-level Approach ($b = 3$): * Partition L_1 into blocks: $B_1 = (1, 2, 3), B_2 = (8, 9, 15), B_3 = (20, 35, 40)$. * Meta-array $L'_1 = (1, 8, 20)$ (First elements). * **Merge** L'_1 with $L_2 = (2, 5, 35)$: * 2 falls in range of B_1 ($1 \leq 2 < 8$). Check B_1. $(1, 2, 3) \cap \{2\} \rightarrow$ **Found 2**. * 5 falls in range of B_1. Check B_1. Not found. * 35 falls in range of B_3 ($20 \leq 35$). Check B_3. $(20, 35, 40) \cap \{35\} \rightarrow$ **Found 35**. Q1: List Intersection|Back to Question

---

Sol 23/06/25 Q2

Analysis of H/L: * H has 11 ones $\implies n = 11$ items. * H length is 22 bits $\implies n + \lfloor u/2^l \rfloor = 22 \implies u/2^l = 11$. * L has 22 bits. Since $n = 11$, lower bits $l = 22/11 = 2$. * Upper bits $h = \lceil \log(u/2^l) \rceil$? From H structure, bucket count is 16 $(0 \ldots 15)$. $h = 4$. * Total bits $b = l + h = 6$. Universe $U \approx 2^6 = 64$.

1. Decompression: * Iterate through H (unary buckets) + L (fixed 2 bits). * H: `1` (bucket 0), `1` (bucket 0), `0`... * Items: 1. B0 (0000) + L_1 (`01`) $\rightarrow$ 000001 = 1. 2. B0 (0000) + L_2 (`11`) $\rightarrow$ 000011 = 3. 3. B1 (0001) + L_3 (`00`) $\rightarrow$ 000100 = 4. 4. ... * 8 (1000 01), 9 (1001 00), 16 (10000 00). * $23, 27, 31, 40$ (Wait, 40? Last item).

2. Access(8): * We want the 8th item (index 8). * Find 8-th `1` in H. It's at position 14. * `Select_1(H, 8) = 14`. * Value of High part: `pos - rank` or `pos - index`. * Number of 0s before position 14: $14 - 8 = 6$. High part = 6 (`0110`). * Low part: Read $L[8]$ (2 bits) = `11`. * Result: `0110 11` = 27.

3. NextGEQ(31): * Target $x = 31$. Binary 011111 (6 bits). * High $x_h = 7$ (`0111`), Low $x_l = 3$ (`11`). * Find bucket 7 in H: `Select_0(H, 7)`. * This gives start of bucket 7. Check items in bucket 7. * If bucket 7 empty or all items < 31, go to bucket 8. * From table, Bucket 7 contains item with low `00` (28). $28 < 31$. * Next item is in Bucket > 7. * Found item 31? Yes, matches. Q2: Elias-Fano Operations|Back to Question

---

Sol 23/06/25 Q3

Structure Construction: * B len 24. $Z = 6$ (Superblock), $z = 2$ (Block). * **Superblocks (R_s):** Store absolute rank at $i = 1, 7, 13, 19$. * $R_s[0] = 0$. * $R_s[1] = \text{popcount}(B[1..6]) = 4$ (`01 11 10`). * $R_s[2] = 4 + \text{popcount}(B[7..12]) = 4 + 3 = 7$ (`01 01 01`). * $R_s[3] = 7 + \text{popcount}(B[13..18]) = 7 + 4 = 11$ (`11 11 00`). * **Blocks (R_b):** Store relative rank within superblock. Size 2. * Reset at every superblock. * SB0: `[0, 1, 3]`. (Offsets for pos 1, 3, 5). * SB1: `[0, 1, 2]`. * ...

Rank(15): * $i = 15$. * Superblock index: $\lfloor (15-1)/6 \rfloor = 2$. Base Rank $R_s[2] = 7$. * Block index within SB: $\lfloor (15-13)/2 \rfloor = 1$. Block Rank $R_b[\ldots] = 2$ (bits in `11` at 13-14). * In-block: check bits 15. $B[15] = 0$. Popcount is 0. * Total: $7 + 2 + 0 = 9$. * *Correction from text:* Hand calculation in note says 10. Let's re-check B string. * `01 11 10` (4) * `01 01 01` (3) -> Sum 7. * `11 11 00` -> 13,14 are 1. 15 is 0. * Rank(14) is $7 + 2 = 9$. Rank(15) is 9. * *Handwritten note says 10?* Ah, `11` at pos 13,14. Rank at 12 is 7. 13(1), 14(1). Rank(14)=9. Q3: Rank Data Structure|Back to Question

---

Sol 23/06/25 Q4

Algorithm: Iterate through nodes. Check Parent, then Parent of Parent. * Assume LOUDS encoding. B describes tree structure. * `Select_1(B, i)` finds the position of node i in the bitstream. * `Rank_1` allows moving between children and parents. * **Parent(i):** $p = \text{Select}_0(B, \text{Rank}_1(B, \text{start_node_i}))\ldots$ (Actually LOUDS parent logic: index of the 0 that corresponds to the 1 of the child).

Pseudocode (Simplified from notes):

```
count = 0
for i = 1 to n:
    // Get Parent
    id1 = parent(i) // O(1) via Rank/Select
    if id1 == NULL: continue

    // Get Grandparent
    id2 = parent(id1)
    if id2 == NULL: continue

    // Check Labels
    if L[id1] == "A" AND L[id2] == "B":
        count++
return count
```

Q4: Succinct Tree Algorithms|Back to Question

Exam: 10 January 2025

Q1: Canonical Huffman

Problem: Given symbols and probabilities:

$$p(a) = 0.36, p(b) = 0.18, p(c) = 0.15, p(d) = p(e) = 0.07, p(f) = 0.17$$

1. Construct the **Canonical Huffman code**, showing the step-by-step table construction (Sort by length, then symbol). 2. **Decompress** the first three symbols from the bit sequence: `0111001`. Sol 10/01/25 Q1|See Solution

Q2: Two-Level String Indexing

Problem: Given the ordered set of strings:

$$S = \{\text{AAAA, AAAB, AADA, AADB, AADC, AADD, AADFA, BB}\}$$

1. Build a **two-level indexing scheme**: * **Disk:** Blocks of 2 strings, Front-Coded. * **Memory:** Patricia Trie (PT) indexing the first string of every block. 2. Show how to search for all patterns prefixed by $P = \text{AAD}$. Sol 10/01/25 Q2|See Solution

Q3: Integer Compression

Problem: 1. Write the first 7 codewords of the **(s,c)-dense code** with $s = 1$ and $c = 3$. 2. Compress the integers **4, 6, 11** in the sequence $S = (1, 4, 5, 6, 9, 11, 15)$ using **Interpolative Coding**. Sol 10/01/25 Q3|See Solution

Q4: Rank Data Structure

Problem: Given the binary array:

$$B[1, 24] = 01\ 11\ 00\ 01\ 01\ 00\ 11\ 11\ 00\ 11\ 00\ 11$$

1. Build the $O(1)$ Rank data structure ($Z = 6, z = 2$). 2. Show the execution of **Rank(15)**. Sol 10/01/25 Q4|See Solution

Q5: Succinct Trees

Problem: Given a binary tree defined by edges $(a, b)(b, c)(b, d)(c, e)(c, f)$. 1. Encode it succinctly (e.g., LOUDS or BP). 2. Verify algorithmically if the path $[a, g]$ exists. Sol 10/01/25 Q5|See Solution

---

Exam: 10 February 2023

Q1: Sampling Simulation

Problem: Given sequence $S = [a, b, c, d, e, f, g, h, i, l]$. Simulate **Reservoir Sampling** with $m = 2$. 1. **Known Length ($n = 10$):** Use random probabilities $p = [0.5, 0.5, 0.5, 1, 1, 0.1, 0.5, 1, 0.1, 1]$. 2. **Unknown Length:** Use random integers $h = [1, 3, 4, 2, 1, 5, 4, 6]$. Sol 10/02/23 Q1|See Solution

Q2: Uncompacted Trie & Succinctness

Problem: Given strings $S = \{\text{AB, ACA, ACB, CA, CB}\}$. 1. Build the **Uncompacted Trie** (Alphabet $\Sigma = \{A, B, C\}$). 2. Show the succinct encoding (LOUDS bitvector B). 3. Write pseudocode to find the **length of the leftmost path** using only B and Rank/Select. Sol 10/02/23 Q2|See Solution

Q3: Treap Operations

Problem: Build a **Min-Treap** by inserting the following pairs:

$$\langle E, 1\rangle, \langle C, 14\rangle, \langle M, 5\rangle, \langle A, 12\rangle, \langle B, 8\rangle$$

(Order: Alphabetical for keys). Sol 10/02/23 Q3|See Solution

Q4: Arithmetic Decoding

Problem: Decode the compressed sequence $\langle 4 \text{ chars}, 011110\rangle$. **Model:** $P(a) = 1/4, P(c) = 1/4, P(b) = 1/2$. Sol 10/02/23 Q4|See Solution

Solutions: 10 January 2025

Sol 10/01/25 Q1

1. Huffman Construction: * Probabilities: $d(0.07), e(0.07), c(0.15), f(0.17), b(0.18), a(0.36)$. * Merge $d + e \to de(0.14)$. * Merge $de + c \to dec(0.29)$. * Merge $f + b \to fb(0.35)$. * Merge $dec + fb \to decfb(0.64)$. * Merge $a + decfb \to Root(1.0)$. * **Lengths:** $a(1), f(3), b(3), c(3), d(4), e(4)$. * **Canonical Codes (Sort len, then symbol):** * $L = 1$: `a` $\to$ `0`. * $L = 3$: `b, c, f`. $FC[3] = (FC[1] + 1) \ll 2$? No. * **FC Calculation:** * $L = 4$: `d, e`. $FC[4] = 0000$ (start). `d=0000, e=0001`. Next avail: `0010`. * $L = 3$: `b, c, f`. $FC[3] = 0010 \gg 1 = 001$. * Codes: `b=001, c=010, f=011`. Next avail: `100`. * $L = 1$: `a`. $FC[1] = 100 \gg 2 = 1$. Code: `a=1`. * **Final Table:** $a \to 1, b \to 001, c \to 010, f \to 011, d \to 0000, e \to 0001$. (Note: Handwritten notes use slightly different FC logic, result matches lengths).

2. Decoding `0111001`: * `0`: a? No, a is `1`. Wait, handwritten notes have a at length 1? * Let's re-read the bitstream with the generated codes. * `011`: Matches `f`. Output **f**. Rem: `1001`. * `1`: Matches `a`. Output **a**. Rem: `001`. * `001`: Matches `b`. Output **b**. * **Result:** `fab`. Q1: Canonical Huffman|Back to Question

---

Sol 10/01/25 Q2

1. Indexing Scheme: * Blocks ($B = 2$): * B_1: `AAAA`, `AAAB`. Sep: `AAAA`. * B_2: `AADA`, `AADB`. Sep: `AADA`. * B_3: `AADC`, `AADD`. Sep: `AADC`. * B_4: `AADFA`, `BB`. Sep: `AADFA`. * **Patricia Trie:** Insert separators `AAAA, AADA, AADC, AADFA`. * `AA` common to all. Edge `(A,A)`. * Node splits at chars `A` (from `AAAA`), `D` (from `AAD`...). * Structure: Root $\xrightarrow{AA}$ Node1 $\xrightarrow{A}$ Leaf(`AAAA`). Node1 $\xrightarrow{D}$ Node2 $\xrightarrow{A}$ Leaf(`AADA`). Node2 $\xrightarrow{C}$ Leaf(`AADC`). Node2 $\xrightarrow{F}$ Leaf(`AADFA`).

2. Search `AAD`: * Search `AAD$` and `AAD#` in Patricia. * `AAD$`: Matches path to `AADA`? Mismatch. Falls before `AADA`. * `AAD#`: Matches path to `AADFA`. * Range covers blocks B_2 (starts `AADA`), B_3 (starts `AADC`). * Retrieve and scan B_2, B_3. Q2: Two-Level String Indexing|Back to Question

---

Sol 10/01/25 Q3

1. (s,c)-dense code ($s = 1, c = 3$): * $2^1 + 2^3$ not valid? Dense code usually sums to 2^k. Here $1 + 3 = 4 = 2^2$. * Stoppers ($s = 1$): `0`. * Continuers ($c = 3$): `10, 110, 111`? Or `100, 101, 110`? * Handwritten table shows: * `00`, `01` (Wait, sequence values). * Codes: `0` (stopper), `10`, `11` (continuers? No). * Correct logic: $k = 2$. $S = 1$ stopper `00`? No. * Standard (s,c): Stopper is suffix of length $\log s$? * Notes: "S=1 means 1 stopper `00`". "C=3 means 3 continuers `01, 10, 11`". * **Codewords:** `00`, `01`, `10`, `11`...

2. Interpolative (4, 6, 11): * Seq S: `1, 4, 5, 6, 9, 11, 15`. Range $[1, 15]$. $n = 7$. * **Encode 6 (Index 4):** * $m = 4$. $S[4] = 6$. * Low bound: $1 + (4 - 1) = 4$. High bound: $15 - (7 - 4) = 12$. * Range $[4, 12]$. Encode $6 - 4 = 2$. Size 9. Bits $\lceil \log 9 \rceil = 4$. * Code: `0010`. * **Recurse Left (Encode 4, Index 2):** * Range $[1, 3]$ for indices $1, 2, 3$. * $S[2] = 4$. Low $1 + (2 - 1) = 2$. High $6 - (4 - 2) = 4$. * Range $[2, 4]$. Encode $4 - 2 = 2$. Size 3. Bits 2. * Code: `10`. * **Recurse Right (Encode 11, Index 6):** * Range for $9, 11, 15$. * $S[6] = 11$. Low $6 + (6 - 5) = 7$. High $15 - (7 - 6) = 14$. * Range $[7, 14]$. Encode $11 - 7 = 4$. * Code: `100`. Q3: Integer Compression|Back to Question

---

Solutions: 10 February 2023

Sol 10/02/23 Q1

1. Known Length ($n = 10, m = 2$): * Formula: Pick $S[j]$ if $p < \frac{m-\text{selected}}{n-j+1}$. * $j = 1$ ('a'): $p = 0.5$. Threshold $2/10 = 0.2$. $0.5 > 0.2$. No. * $j = 2$ ('b'): $p = 0.5$. Threshold $2/9 \approx 0.22$. $0.5 > 0.22$. No. * ... (Simulation continues). * *Notes show:* f and i selected.

2. Unknown Length: * h values: `[1, 3, 4, 2, 1, 5, 4, 6]`. * Init: `R = [a, b]`. * $j = 3$ ('c'): $h = 1 \leq 2$. Swap $R[1] \leftarrow c$. $R = [c, b]$. * $j = 4$ ('d'): $h = 3 > 2$. Skip. * $j = 5$ ('e'): $h = 4 > 2$. Skip. * $j = 6$ ('f'): $h = 2 \leq 2$. Swap $R[2] \leftarrow f$. $R = [c, f]$. * ... Q1: Sampling Simulation|Back to Question

Sol 10/02/23 Q2

1. Uncompacted Trie: * Root * A → B (leaf), C → A (leaf), B (leaf). * C → A (leaf), B (leaf). * 9 nodes total.

2. Succinct Encoding (LOUDS): * Level-order traversal (BFS). * `10` (Root, 2 children A, C). * `110` (Node A, 2 children B, C). * `110` (Node C, 2 children A, B). * Leaves: `0 0 0 0`. * $B = 10\ 110\ 110\ 0\ 0\ 0\ 0\ 0$. (Note: LOUDS usually starts with `10` for super-root).

3. Left-path Algorithm:

```
len = 0; i = 2; // Start at root's first child
while (B[i] == 1) { // While child exists
    len++;
    i = select_1(B, rank_1(B, i) + 1); // Go to first child of current node
    // Wait, standard LOUDS child navigation:
    // Child k of node at pos i is at select_0(B, rank_1(B, i) + k - 1) + 1?
    // Notes use: i = select_0(B, rank_1(B, i)) + 1 (First child)
}
return len;
```

Q2: Uncompacted Trie & Succinctness|Back to Question

Sol 10/02/23 Q3

Treap Insertion: 1

- **Insert (E, 1):** Root (E, 1).
- **Insert (C, 14):** Left of E.
- **Insert (M, 5):** Right of E.
- **Insert (A, 12):** Left of C.
- **Insert (B, 8):** Right of A? No, Left of C, Right of A.
  - $B > A$, $B < C$. $8 < 12$. Rotate B up over A.
  - $B < C$. $8 < 14$. Rotate B up over C.
  - Subtree: $E \rightarrow (Left)B \rightarrow (Left)A, (Right)C$.

Q3: Treap Operations|Back to Question

Sol 10/02/23 Q4

Arithmetic Decoding: 2

- **Ranges:**
  - $a[0, 0.25)$.

- $c[0.25, 0.5)$.
- $b[0.5, 1.0)$.

- **Bits:** `011110...` ≈ 0.46875.
- **Step 1:** 0.46875 in $[0.25, 0.5)$? Yes. **Symbol: c**.
  - Zoom: $New = (0.468 - 0.25)/0.25 = 0.875$.
- **Step 2:** 0.875 in $[0.5, 1.0)$? Yes. **Symbol: b**.
  - Zoom: $(0.875 - 0.5)/0.5 = 0.75$.
- **Step 3:** 0.75 in $[0.5, 1.0)$? Yes. **Symbol: b**.
  - Zoom: $(0.75 - 0.5)/0.5 = 0.5$.
- **Step 4:** 0.5 in $[0.5, 1.0)$? Yes. **Symbol: b**.
- Result: cbbb.

Q4: Arithmetic Decoding|Back to Question

Exam: 06 November 2023 (Midterm)

Q1: Snow Plow Simulation

Problem: Simulate the **Snow Plow** algorithm on the sequence:

$$S = (2, 6, 5, 4, 1, 7, 3, 8, 1, 4, 2)$$

* **Memory:** $M = 3$ items. * Show the formation of runs and the content of the Heap/Unsorted buffer. Sol 06/11/23 Q1|See Solution

Q2: Treap Construction

Problem: Build a **Min-Treap** (Key, Priority) by inserting the following pairs in order:

$$S = \{\langle D, 4\rangle, \langle A, 5\rangle, \langle G, 9\rangle, \langle B, 3\rangle, \langle F, 6\rangle, \langle E, 2\rangle\}$$

* **Key:** 1st component (Letter). * **Priority:** 2nd component (Integer). Sol 06/11/23 Q2|See Solution

Q3: MOPH Construction

Problem: Given 4 strings $S = \{11, 22, 33, 44\}$. Construct a **Minimal Ordered Perfect Hash Function** (MOPHF). * **Hash Functions:** * $h_1(xy) = x + 3y \pmod 7$ * $h_2(xy) = x + 2y \pmod 7$ * x, y are the digits (e.g., for "11", $x = 1, y = 1$). * **Target:** $h(k) = (g(h_1(k)) + g(h_2(k))) \pmod 4$. Sol 06/11/23 Q3|See Solution

---

Exam: 15 January 2024

Q1: Two-Level String Indexing

Problem: Given the set of strings:

$$S = \{\text{BAA, BAB, BACAA, BACAB, BACAD, BACB, CA, CB}\}$$

1. Index S via a **two-level scheme**: * Block size $B = 2$ strings. * Patricia Trie in internal memory. 2. Show the steps for: * Lexicographic search of `BB`. * Prefix search of `BAC`. Sol 15/01/24 Q1|See Solution

Q2: Multi-key Quicksort Trace

Problem: Sort the sequence of strings:

$$S = (\text{BACAB, ABB, BBC, DD, DF})$$

* **Pivot Rule:** Always pick the **first string** of the subsequence as pivot. Sol 15/01/24 Q2|See Solution

Q3: Compression Pipeline

Problem: Given the text $T = \text{ABRABRA}$. Apply the pipeline: **BWT** $\rightarrow$ **MTF** $\rightarrow$ **RLE0** (Wheeler) $\rightarrow$ **Arithmetic Coding** (on the first 3 numbers). * **MTF:** Initial list $\{A, B, R\}$. Sol 15/01/24 Q3|See Solution

Solutions: 06 November 2023

Sol 06/11/23 Q1

Trace with M=3: 1. **Load:** `2, 6, 5`. Heap $H = [2, 5, 6]$. Unsorted $U = []$. 2. **Output 2:** Read `4`. $4 > 2 \rightarrow H = [4, 5, 6]$. 3. **Output 4:** Read `1`. $1 < 4 \rightarrow U = [1]$. $H = [5, 6]$. 4. **Output 5:** Read `7`. $7 > 5 \rightarrow H = [6, 7]$. $U = [1]$. 5. **Output 6:** Read `3`. $3 < 6 \rightarrow U = [1, 3]$. $H = [7]$. 6. **Output 7:** Read `8`. $8 > 7 \rightarrow H = [8]$. $U = [1, 3]$. 7. **Output 8:** Read `1`. $1 < 8 \rightarrow U = [1, 3, 1]$. $H = []$. * **End of Run 1:** `2, 4, 5, 6, 7, 8`. * **Restart:** Load U to H. $H = [1, 1, 3]$. 8. **Output 1:** Read `4`. $4 > 1 \rightarrow H = [1, 3, 4]$. 9. **Output 1:** Read `2`. $2 > 1 \rightarrow H = [2, 3, 4]$. 10. **Output 2:** Empty input. 11. **Output 3, 4.** * **End of Run 2:** `1, 1, 2, 3, 4`. Q1: Snow Plow Simulation|Back to Question

---

Sol 06/11/23 Q2

Insertions (Min-Heap Priority): 1. **<D, 4>:** Root. 2. **<A, 5>:** $A < D$ (Left). Prio $5 > 4$. Left Child of D. 3. **<G, 9>:** $G > D$ (Right). Prio $9 > 4$. Right Child of D. 4. **<B, 3>:** $B < D$, $B > A$. Insert Right of A. * Prio $3 < 5$ (Swap with A). * Prio $3 < 4$ (Swap with D). * **New Root: B**. Left: A. Right: D. 5. **<F, 6>:** $F > B$, $F > D$, $F < G$. Left of G. * Prio $6 < 9$ (Swap with G). Right of D becomes F. 6. **<E, 2>:** $E > B$, $E > D$, $E < F$. Left of F. * Prio $2 < 6$ (Swap F). * Prio $2 < 4$ (Swap D). * Prio $2 < 3$ (Swap B). * **New Root: E**.

Final Structure: * Root: **E (2)** * Left: **B (3)** $\rightarrow$ Left: **A (5)**. Right: **D (4)**. * Right: **F (6)** $\rightarrow$ Right: **G (9)**. Q2: Treap Construction|Back to Question

---

Sol 06/11/23 Q3

Hash Values: * **11:** $h_1 = 1 + 3 = 4$. $h_2 = 1 + 2 = 3$. Edge $(4, 3)$. * **22:** $h_1 = 2 + 6 \equiv 1$. $h_2 = 2 + 4 = 6$. Edge $(1, 6)$. * **33:** $h_1 = 3 + 9 \equiv 5$. $h_2 = 3 + 6 \equiv 2$. Edge $(5, 2)$. * **44:** $h_1 = 4 + 12 \equiv 5$. $h_2 = 4 + 8 \equiv 5$. Loop $(5, 5)$. * **Impossible:** MOPHF graph must be acyclic. Key `44` creates a self-loop (or collision if mapped to edge). Wait, MOPHF on graph requires distinct endpoints? Or just solving equations? * Equation: $g[5] + g[5] = \text{Rank}(44) \pmod 4 \implies 2g[5] = 3 \pmod 4$. * $2x \equiv 3 \pmod 4$ has **no solution** (LHS is even, RHS is odd). * **Result:** Impossible to build. Q3: MOPH Construction|Back to Question

---

Solutions: 15 January 2024

Sol 15/01/24 Q1

1. Two-Level Structure: * **Blocks (size 2):** * B_1: `BAA`, `BAB`. (Sep: `BAA`) * B_2: `BACAA`, `BACAB`. (Sep: `BACAA`) * B_3: `BACAD`, `BACB`. (Sep: `BACAD`) * B_4: `CA`, `CB`. (Sep: `CA`) * **Patricia Trie:** Keys `BAA, BACAA, BACAD, CA`. *

Root splits on ‘B’ vs ‘C’. * ‘B’ branch splits at 3rd char (‘A’ vs ‘C’). * ‘C’ branch (from `BAC...`) splits at 4th char (‘A’ vs ‘A’ vs ‘D’??).

2. Search `BB`: * Trie search for `BB`. Stops at node distinguishing `BA...` vs `CA`. * Matches `BA...` prefix? No, `B` matches, then `A` vs `B`. * Finds predecessor `BAA` (or `BAC...` depending on trie structure). * Moves to Block 3? No, `BB` > `BAC...`. * Trace: `BB` is between `BACB` (B_3) and `CA` (B_4). * Search in B_3. Not found. Lexicographically after B_3.

3. Prefix Search `BAC`: * Search `BAC$` and `BAC#`. * `BAC$`: Matches `BACAA` (B_2). * `BAC#`: Matches `BACB` (B_3). * Result: Spans from B_2 to B_3. Return all strings in these blocks that match prefix. Q1: Two-Level String Indexing|Back to Question

---

Sol 15/01/24 Q2

Trace: Sequence: `BACAB, ABB, BBC, DD, DF`. 1. **Pivot:** `BACAB`. * $S_<$: `ABB`. * $S_=$: `BACAB`, `BBC` (Wait, `B` match, `B` vs `A`?). * Pivot char 0: ‘B’. * `ABB` (‘A’ < ‘B’) → $S_<$. * `BBC` (‘B’ == ‘B’) → $S_=$. * `DD` (‘D’ > ‘B’) → $S_>$. * `DF` (‘D’ > ‘B’) → $S_>$. * **Result:** $S_< = \{ABB\}$. $S_= = \{BACAB, BBC\}$. $S_> = \{DD, DF\}$. 2. **Recurse $S_=$:** $\{BACAB, BBC\}$. Pivot `BACAB`. Char 1: ‘A’. * `BBC`: ‘B’ > ‘A’ → $S_>$. * Result: `BACAB`, then `BBC`. 3. **Recurse $S_>$:** $\{DD, DF\}$. Pivot `DD`. Char 1: ‘D’. * `DF`: ‘F’ > ‘D’ → $S_>$. * Result: `DD`, then `DF`. **Final Order:** `ABB, BACAB, BBC, DD, DF`. Q2: Multi-key Quicksort Trace|Back to Question

---

Sol 15/01/24 Q3

Pipeline: T = ABRABRA. 1. **BWT:** * Matrix rotations. Sort: `ABRA...`, `ABRA...`, `BRA...`, `BRA...`, `RA...`, `RA...`. * Last Column L: `ARD...`? * Hand calculation: `ABRABRA` → `ABRAABR`? * Notes say: `L = ARRAABB`. 2. **MTF (List A,B,R):** * `A`: idx 0. List `A,B,R`. Out: 0. * `R`: idx 2. List `R,A,B`. Out: 2. * `R`: idx 0. List `R,A,B`. Out: 0. * `A`: idx 1. List `A,R,B`. Out: 1. * `A`: idx 0. List `A,R,B`. Out: 0. * `B`: idx 2. List `B,A,R`. Out: 2. * `B`: idx 0. List `B,A,R`. Out: 0. * **MTF Out:** `0, 2, 0, 1, 0, 2, 0`. (Note: Handwritten solution might differ slightly, cites `1, 3, 1...` if 1-based). 3. **RLE0 (Wheeler):** * `0` → Encoded. * `2` → 2+1 = 3. * `0` → Encoded. * `1` → 2. * Output: `3, 1, 3, 1...` (Based on notes). 4. **Arithmetic:** Encode first 3 numbers. Q3: Compression Pipeline|Back to Question

---

Exam: 11 July 2024

Q1: Reservoir Sampling Trace

Problem: Simulate **Reservoir Sampling** with $m = 3$ items from a sequence of length $n = 9$:

$$S = [a, b, c, d, e, f, g, h, i]$$

Random Stream: The algorithm extracts the following random integers h for steps $i = 4 \dots 9$:

$$H = [2, 4, 1, 2, 3, 1]$$

Show the content of the Reservoir R at each step. Sol 11/07/24 Q1|See Solution

Q2: Treap Split & Merge

Problem: 1. Build a **Min-Treap** with pairs $\langle \text{Key}, \text{Priority} \rangle$:

$$S = \{\langle C, 4\rangle, \langle A, 6\rangle, \langle I, 10\rangle, \langle B, 3\rangle, \langle H, 7\rangle, \langle F, 2\rangle\}$$

(Insert in this order). 2. Execute **SPLIT** on key D. 3. Execute **MERGE** on the two resulting Treaps. Sol 11/07/24 Q2|See Solution

Q3: Patricia Trie Search

Problem: Given strings $S = \{\text{abaa, abca, abma, baa, bbb}\}$. 1. Build the **Patricia Trie**. 2. Show the steps for **Lexicographic Search** of $P_1 = \text{bbc}$. 3. Show the steps for search of $P_2 = \text{abb}$. Sol 11/07/24 Q3|See Solution

Q4: Arithmetic Coding (Dyadic)

Problem: Encode $T = \text{aba}$ using Arithmetic Coding. * **Probabilities:** $P(a) = 3/4, P(b) = 1/4$. * **Hint:** Work with **dyadic fractions** (e.g., $3/4, 9/16$), do not convert to decimals. Sol 11/07/24 Q4|See Solution

---

Exam: 4 February 2025

Q1: Snow Plow Simulation

Problem: Simulate **Snow Plow** on the sequence:

$$S = (2, 5, 4, 3, 1, 4, 2, 3, 5)$$

* **Memory:** $M = 3$. * Show the runs generated. Sol 04/02/25 Q1|See Solution

Q2: Multi-key Quicksort

Problem: Sort the set:

$$S = \{\text{AADD, BB, AADFA, AADB, AAAA, AAAB, AADA}\}$$

* **Pivot Rule:** Always choose the **first string** of the subset. Sol 04/02/25 Q2|See Solution

Q3: Elias-Fano Compression

Problem: Given integers $S = (1, 2, 3, 6, 8, 10, 11, 15, 19, 23, 31)$. 1. Compress via **Elias-Fano** (Show calculations for u, n, l, h). 2. Show execution of **Access(5)** (indices start from 1). 3. Show execution of **NextGEQ(9)**. Sol 04/02/25 Q3|See Solution

Q4: Arithmetic Decoding

Problem: * **Model:** $p(a) = 1/2, p(b) = 1/4, p(c) = 1/8, p(d) = 1/8$. * **Input:** Bit sequence `111101...` * **Task:** Decompress the first **two symbols**. Sol 04/02/25 Q4|See Solution

---

Exam: 9 December 2024

Q1: Integer Coding Variety

Problem: 1. Compress numbers **2** and **15** using **Rice Code** ($k = 2$). 2. Write the first 12 codewords of the **(6, 2)-dense code**. 3. Compress **4, 6, 10** in sequence $S = (3, 4, 5, 6, 9, 10, 16)$ using **Interpolative Coding**. Sol 09/12/24 Q1|See Solution

Q2: Elias-Fano Decode

Problem: * $L = 01\ 11\ 00\ 01\ 01\ 00\ 11\ 11\ 00\ 11\ 00$ * $H = 110\ 110\ 10\ 0\ 10\ 10\ 10\ 110\ 0\ 0\ 10\ 0\ 0\ 0\ 0\ 0$ * **Task:** 1. **Access(7)**. 2. **NextGEQ(31)**. Sol 09/12/24 Q2|See Solution

Solutions

Solutions: 11 July 2024

Sol 11/07/24 Q1

Reservoir Sampling Trace: $m = 3, n = 9$. 1. **Init:** $R = [a, b, c]$. 2. **Step** $j = 4$ **(d):** Rand $h = 2$. $2 \leq 3$. Swap $R[2] \leftarrow d$. $R = [a, d, c]$. 3. **Step** $j = 5$ **(e):** Rand $h = 4$. $4 > 3$. Skip. 4. **Step** $j = 6$ **(f):** Rand $h = 1$. $1 \leq 3$. Swap $R[1] \leftarrow f$. $R = [f, d, c]$. 5. **Step** $j = 7$ **(g):** Rand $h = 2$. $2 \leq 3$. Swap $R[2] \leftarrow g$. $R = [f, g, c]$. 6. **Step** $j = 8$ **(h):** Rand $h = 3$. $3 \leq 3$. Swap $R[3] \leftarrow h$. $R = [f, g, h]$. 7. **Step** $j = 9$ **(i):** Rand $h = 1$. $1 \leq 3$. Swap $R[1] \leftarrow i$. $R = [i, g, h]$. **Final Reservoir:** $\{i, g, h\}$. Q1: Reservoir Sampling Trace|Back to Question

Sol 11/07/24 Q2

1. Construction: * **(C,4):** Root. * **(A,6):** $A < C$, $6 > 4$. Left child of C. * **(I,10):** $I > C$, $10 > 4$. Right child of C. * **(B,3):** $B < C$, $B > A$. Insert Right of A. Prio $3 < 6$ (Swap A). Prio $3 < 4$ (Swap C). **New Root B**. * **(H,7):** $H > B$, $H > C$, $H < I$. Left of I. Prio $7 < 10$. Swap I. * **(F,2):** $F > B$, $F > C$. Insert Right of C. Prio $2 < 4$ (Swap C). Prio $2 < 3$ (Swap B). **New Root F**. * **Final:** F(2) → Left: B(3) → Left: A(6), Right: C(4). Right: H(7) → Right: I(10).

2. Split on D: * Insert $(D, -\infty)$. Rotates to Root. * $T_{\leq D}$: Left tree of F (B, A, C) + D. * $T_{>D}$: Right tree of F (H, I).

3. Merge: * Create dummy root $(-\infty)$. Left child $T_{\leq D}$, Right child $T_{>D}$. * Sink dummy. (Standard merge restores original if no changes made). Q2: Treap Split & Merge|Back to Question

Sol 11/07/24 Q3

1. Patricia Trie: * Strings: `abaa, abca, abma, baa, bbb`. * Root splits on 'a' vs 'b'. * **'a' edge:** `abaa, abca, abma`. * Common prefix `ab`. Edge `(ab)`. * Split on `a, c, m`. * Leaves: `aa` (`abaa`), `ca` (`abca`), `ma` (`abma`). * **'b' edge:** `baa, bbb`. * Common `b`. Edge `(b)`. * Split on `a, b`. * Leaves: `aa` (`baa`), `bb` (`bbb`).

2. Search `bbc`: * Root: Match `b`? Yes (Right branch). * Edge `b`: Skip 1 char. * Next node splits on `a` vs `b`. Pattern char is `c`. * `c > b`. Follows rightmost pointer? No, mismatch at branching. * Lexicographic position is **after** `bbb`.

3. Search `abb`: * Root: Match `a` (Left). * Edge `ab`. Matches `ab`. * Node splits `a, c, m`. Pattern has `b`. * `a < b < c`. * Position is between `abaa` and `abca`. Q3: Patricia Trie Search|Back to Question

Sol 11/07/24 Q4

Arithmetic Encoding (`aba`): * Ranges: $a \in [0, 3/4)$, $b \in [3/4, 1)$. 1. **Encode `a`:** * Current: $[0, 1)$. * New: $[0, 3/4)$. 2. **Encode `b`:** * Range width $w = 3/4$. * b starts at $3/4$ of range. * Low $= 0 + (3/4 \times 3/4) = 9/16$. * High $= 0 + (3/4 \times 1) = 3/4 = 12/16$. * Interval: $[9/16, 12/16)$. 3. **Encode `a`:** * Range width $w = 3/16$. * a is bottom $3/4$. * Low $= 9/16$. * High $= 9/16 + (3/16 \times 3/4) = 9/16 + 9/64 = 36/64 + 9/64 = 45/64$. * Interval: $[36/64, 45/64)$. **Final:** Any value in $[36/64, 45/64)$. Example midpoint. Q4: Arithmetic Coding (Dyadic)|Back to Question

---

Solutions: 04 February 2025

Sol 04/02/25 Q1

Trace (M=3): 1. **Load:** `2, 5, 4`. Heap $H = [2, 4, 5]$. Unsorted $U = []$. 2. **Out 2:** Read `3`. $3 \geq 2 \rightarrow H = [3, 4, 5]$. 3. **Out 3:** Read `1`. $1 < 3 \rightarrow U = [1]$. $H = [4, 5]$. 4. **Out 4:** Read `4`. $4 \geq 4 \rightarrow H = [4, 5]$. $U = [1]$. 5. **Out 4:** Read `2`. $2 < 4 \rightarrow U = [1, 2]$. $H = [5]$. 6. **Out 5:** Read `3`. $3 < 5 \rightarrow U = [1, 2, 3]$. $H = []$. * **Run 1:** `2, 3, 4, 4, 5`. * **Restart:** Load U to H. $H = [1, 2, 3]$. Read `5`. 7. **Out 1:** Read `5`. $5 \geq 1 \rightarrow H = [2, 3, 5]$. 8. **Out 2, 3, 5.** * **Run 2:** `1, 2, 3, 5`. Q1: Snow Plow Simulation|Back to Question

---

Sol 04/02/25 Q2

Multi-key Quicksort: Set: `AADD, BB, AADFA, AADB, AAAA, AAAB, AADA`. Pivot: `AADD`. Char 0: 'A'. * $S_<$: Empty. * $S_>$: `BB` (Char 'B'). * $S_=$: `AADD, AADFA, AADB, AAAA, AAAB, AADA`. (All start with 'A'). * **Recurse $S_=$ (Char 1 'A'):** All match 'A'. * **Recurse $S_=$ (Char 2):** * Pivot `AADD` ('D'). * $S_<$ ('A' < 'D'): `AAAA, AAAB`. * $S_<$ ('D' == 'D'): `AADD, AADFA, AADB, AADA`. * **Recurse on `AADA...` (Char 3):** * Pivot `AADD`. Char 'D'. * $S_<$ ('A', 'B'): `AADA, AADB`. * $S_<$ ('F'): `AADFA`. * $S_=$: `AADD`. **Final Order:** `AAAA, AAAB, AADA, AADB, AADC?, AADD, AADFA, BB`. Q2: Multi-key Quicksort|Back to Question

---

Sol 04/02/25 Q3

1. Elias-Fano Stats: * $S_{last} = 31$. $n = 11$. * Universe $U > 31$. Let's say 32 (next power of 2). * Lower bits $l = \lfloor \log(32/11) \rfloor = \lfloor \log 2.9 \rfloor = 1$. * Upper bits $h = \lceil \log 32 \rceil - 1 = 5 - 1 = 4$. * H array size: $n + 2^h = 11 + 16 = 27$ bits. * L array size: $n \times l = 11 \times 1 = 11$ bits.

2. Access(5): * Index 5 (1-based). * **High:** `Select_1(H, 5) - 5`. * Find 5th '1' in H. Count zeros before it. * H (from 23/06/25 solution, reusing structure): `110 110 10 0 10 10...` * 5th '1' is at pos 7. Zeros before: 2. * High part = 2 (`0010`). * **Low:** Read $L[5]$. L bits: `01 11 00 01 01....` * 5th entry is `01` (index 4 if 0-based). Let's check list. * $S[5] = 8$. Bin `01000`. High `0100` (4), Low `0`? * The provided H/L in previous solutions match $S = (1, 2, 3, 8, 9...)$. * For this specific S (`1,2,3,6,8...`), $S[5] = 8$. * `Access` logic: Reconstruct $High \cdot 2^l + Low$.

3. NextGEQ(9): * Target $x = 9$. * Split x into x_H, x_L. * Find bucket x_H in H using `Select_0`. * Iterate bucket items. Check if item ≥ 9. * First item ≥ 9 is 10. Return 10. Q3: Elias-Fano Compression|Back to Question

---

Sol 04/02/25 Q4

Arithmetic Decoding: * Ranges: * $a[0, 0.5)$. * $b[0.5, 0.75)$. * $c[0.75, 0.875)$. * $d[0.875, 1.0)$. * Input: `111101`. * **Symbol 1:** * Starts with `1` (≥ 0.5). * `11` (≥ 0.75). * `111` (≥ 0.875). * Must be **d**. * Zoom: Range $[0.875, 1.0)$ width 0.125. * Map input v to $(v - 0.875)/0.125$. * **Symbol 2:** * Input `111101...` is high. * Let's look at bits relative to range. * `111` puts us in d. * Remaining bits `101`. * Relative value: $0.101_2 \approx 0.625$. * 0.625 falls in b range ($[0.5, 0.75)$). * **Symbol: b**. * **Result:** `db`. Q4: Arithmetic Decoding|Back to Question

---

Solutions: 09 December 2024

Sol 09/12/24 Q1

1. Rice (k=2) on 2, 15: * $M = 2^2 = 4$. * **2:** $q = \lfloor (2-1)/4 \rfloor = 0$. $r = 1$. Code: `1` (Unary 0) `01`. $\rightarrow$ `101`. (Assuming $U(0) = 1$). * **15:** $q = \lfloor 14/4 \rfloor = 3$. $r = 2$. Code: `0001` (Unary 3) `10`. $\rightarrow$ `000110`.

2. (6,2)-dense code: * $s = 6$ stoppers, $c = 2$ continuers. Total 8 patterns ($2^3 \rightarrow 3$ bits). * **Stoppers ($0 \ldots 5$):** `000, 001, 010, 011, 100, 101`. * **Continuers ($6 \ldots 7$):** `110, 111`. * First 12 codewords: * 0-5: `000` to `101`. * 6: `110` + `000` (Cont + Stop 0). * 7: `110` + `001`. * ... * 11: `110` + `101`.

3. Interpolative (4, 6, 10) in S: $S = (3, 4, 5, 6, 9, 10, 16)$. Range indices $[1, 7]$. $Low = 3, High = 16$. * **Root ($m = 4$):** $S[4] = 6$. * Bounds: $Low + (4 - 1) = 6$. $High - (7 - 4) = 13$. * Range $[6, 13]$. Encode $6 - 6 = 0$. * Bits: $\lceil \log(13 - 6 + 1) \rceil = 3$. * Code: `000`. * **Left ($m = 2$):** $S[2] = 4$. * Range $[1, 3]$. $Low = 3, High = S[4] - 1 = 5$. * Bounds: $3 + (2 - 1) = 4$. $5 - (3 - 2) = 4$. * Range $[4, 4]$. Size 1. **0 bits**. * **Right ($m = 6$):** $S[6] = 10$. * Range $[5, 7]$. $Low = S[4] + 1 = 7, High = 16$. * Bounds: $7 + (6 - 5) = 8$.

$16 - (7 - 6) = 15$. * Range $[8, 15]$. Encode $10 - 8 = 2$. * Bits: $\lceil \log 8 \rceil = 3$. * Code: `010`. Q1: Integer Coding Variety|Back to Question

---

Exam: 12 December 2023 (Final Term)

Q1: Patricia Trie Search

Problem: Given strings $S = \{\text{abab, abca, abma, baa, bbb}\}$. 1. Build a **Patricia Trie** for S. 2. Show the steps for lexicographic search of $P_1 = \text{aaa}$. 3. Show the steps for lexicographic search of $P_2 = \text{abb}$. Sol 12/12/23 Q1|See Solution

Q2: Integer Compression

Problem: Given integers $S = (2, 3, 4, 5, 6, 10, 11)$. 1. Compress via **(2,6)-dense code**. Show the first 12 codewords for integers $0 \ldots 11$. 2. Compress via **Interpolative Coding** for the subset of numbers: $5, 3, 10$. Sol 12/12/23 Q2|See Solution

Q3: Succinct Tree Encoding

Problem: Given the tree with root "a": * $a \to b$ (right child) * $b \to c$ (left child), $b \to e$ (right child) * $c \to d$ (right child) Show its **succinct encoding** (Binary array B and Labels L). Sol 12/12/23 Q3|See Solution

Q4: Elias-Fano Decoding

Problem: Decode the **6th integer** encoded via Elias-Fano. * $L = 01\ 11\ 00\ 01\ 01\ 00\ 11\ 11\ 00\ 11\ 00$ * $H = 110\ 110\ 10\ 0\ 10\ 10\ 10\ 110\ 0\ 0\ 10\ 0\ 0\ 0\ 0\ 0$ *(Hint: Derive n, l, h first).* Sol 12/12/23 Q4|See Solution

Q5: Compression Pipeline

Problem: Text $T = \text{bababac}$. Apply **BWT** $\to$ **MTF** $\to$ **RLE0** (Wheeler) $\to$ **Arithmetic** (first 3 numbers). Sol 12/12/23 Q5|See Solution

---

Exam: 16 January 2023

Q1: Snow Plow Simulation

Problem: Sequence $2, 5, 4, 3, 1, 4, 2$. Memory $M = 2$. Simulate **Snow Plow** and show the runs. Sol 16/01/23 Q1|See Solution

Q2: Patricia Trie & Search

Problem: $S = \{\text{AABA, AACAAAC, AACAACC, BABAA, BABBB, BACA}\}$. 1. Build the **Patricia Trie**. 2. Show search for pattern $P = \text{AACBACD}$. Sol 16/01/23 Q2|See Solution

Q3: Minimal Ordered Perfect Hash

Problem: $S = \{\text{AA, AC, BB, CC}\}$. * Hash Functions: $h_1(xy) = x + y \pmod 7$, $h_2(xy) = x + 2y \pmod 7$. * Codes: $A = 1, B = 2, C = 3$. Construct the MOPHF. Sol 16/01/23 Q3|See Solution

Q4: BWT Pipeline

Problem: $T = \text{ABABAC}$. Pipeline: **BWT** $\to$ **MTF** $\to$ **RLE0** (Wheeler) $\to$ **Huffman**. Sol 16/01/23 Q4|See Solution

---

Exam: 02 February 2022

Q1: LZ77 and LZ78

Problem: String T = abababc. 1. Compress by **LZ77**. 2. Compress by **LZ78**, showing the Trie. Sol 02/02/22 Q1|See Solution

Q2: Full Compression Pipeline

Problem: String S = amata$. Pipeline: **BWT** → **MTF** → **RLE0** → **Huffman**. Sol 02/02/22 Q2|See Solution

Q3: Intersection Algorithms

Problem: $L_1 = (1, 2, 4, 6, 9, 10, 15, 18, 20)$ $L_2 = (2, 3, 7, 8, 18)$ 1. **Mutual Partitioning** intersection. 2. **Two-level storage** intersection ($b = 3$ for L_1). Sol 02/02/22 Q3|See Solution

Solutions

Solutions: 12 December 2023

Sol 12/12/23 Q1

1. Trie Construction: * Keys: `abab, abca, abma, baa, bbb`. * Root splits on 'a' vs 'b'. * **'a' branch:** `abab, abca, abma`. * Common `ab`. Edge `(ab)`. * Node splits on `a, c, m`. * `a` → `b` (leaf `abab`). * `c` → `a` (leaf `abca`). * `m` → `a` (leaf `abma`). * **'b' branch:** `baa, bbb`. * Node splits on `a` vs `b`. * `a` → `a` (leaf `baa`). * `b` → `b` (leaf `bbb`). * **Patricia:** Edge labels are compressed. e.g., `(ab)` edge, `(aa)` edge.

2. Search P_1 = aaa: * Root: Match 'a' (Left). * Edge `ab`. Pattern has `aa`. Mismatch at 2nd char ('b' vs 'a'). * Mismatch index $i = 1$ (0-based, `a` matches `a`). * Trie navigation stops. Compare P_1 with a leaf in this subtree (e.g., `abab`). * $LCP(\text{aaa}, \text{abab}) = 1$. * Mismatch 'a' vs 'b'. 'a' < 'b'. * P_1 is lexicographically **before** `abab`. (Position 0).

3. Search P_2 = abb: * Root: Match 'a'. * Edge `ab`. Pattern `ab`. Match. * Node splits `a, c, m`. Pattern next char `b`. * `a < b < c`. * Position is **between** `abab` (edge `a`) and `abca` (edge `c`). Q1: Patricia Trie Search|Back to Question

Sol 12/12/23 Q2

1. (2,6)-dense code: * $s = 2$ stoppers, $c = 6$ continuers. $2 + 6 = 8 = 2^3$ (3 bits). * **Stoppers** $(0, 1)$**:** `000, 001`. * **Continuers** $(2 \ldots 7)$**:** `010, 011, 100, 101, 110, 111`. * **Codewords:** * 0: `000` * 1: `001` * 2: `010 000` (Cont 0 + Stop 0) * 3: `010 001` (Cont 0 + Stop 1) * 4: `011 000` ...

2. Interpolative (5, 3, 10): Seq $S = (2, 3, 4, 5, 6, 10, 11)$. Range $n = 7$. * **Root:** $m = 4$. $S[4] = 5$. * Range $[1 + 4 - 1, 11 - 3 + 0] = [4, 8]$. * Encode $5 - 4 = 1$ in $\lceil \log 5 \rceil = 3$ bits. Code `001`. * **Left:** Encode 3. $S[2] = 3$. * Range $[1 + 2 - 1, 5 - 2 + 0] = [2, 3]$. * Encode $3 - 2 = 1$ in 1 bit. Code `1`. * **Right:** Encode 10. $S[6] = 10$. * Range $[5 + 2 - 1, 11 - 1 + 0] = [6, 10]$. * Encode $10 - 6 = 4$ in 3 bits. Code `100`. Q2: Integer Compression|Back to Question

Sol 12/12/23 Q3

Tree Encoding: * Edges: $a \to b(R)$, $b \to c(L), e(R)$, $c \to d(R)$. * Structure: * Root `a` has NO Left, YES Right (`b`). * `b` has YES Left (`c`), YES Right (`e`). * `c` has NO Left, YES Right (`d`). * `e` has NO children. * `d` has NO children. * **Succinct BP (Balanced Parentheses):** `(( ))` logic. * **Succinct LOUDS:** * a: `01` (0 left, 1 right? No, LOUDS is degree). * Let's assume degree-based: * a: degree 1 (R). `10`. * b: degree 2. `110`. * c: degree 1. `10`. * e: degree 0. `0`. * d: degree 0. `0`. * Bitvector: `10 110 10 0 0`. Q3: Succinct Tree Encoding|Back to Question

Sol 12/12/23 Q4

Decoding: * $H = 11011010...$ 16 ones $\implies n = 16$. * H len $= 32$? * Calculate l, h. * Access(6). * Find 6th one in H at pos p. * High part $= p - 6$. * Low part $= L[6]$. * Combine. Q4: Elias-Fano Decoding|Back to Question

---

Solutions: 16 January 2023

Sol 16/01/23 Q1

Snow Plow (M=2): 1. Load `2, 5`. $H = [2, 5]$. 2. Out `2`. Read `4`. $4 \geq 2 \to H = [4, 5]$. 3. Out `4`. Read `3`. $3 < 4 \to U = [3]$. $H = [5]$. 4. Out `5`. Read `1`. $1 < 5 \to U = [3, 1]$. $H = []$. * **Run 1:** `2, 4, 5`. 5. Restart $H = [1, 3]$. Read `4`. 6. Out `1`. Read `4`. $H = [3, 4]$. 7. Out `3`. Read `2`. $U = [2]$. $H = [4]$. 8. Out `4`. * **Run 2:** `1, 3, 4`. 9. **Run 3:** `2`. Q1: Snow Plow Simulation|Back to Question

Sol 16/01/23 Q2

Patricia Search `AACBACD`: 1. Root $\to$ Match `A`. 2. Edge `AC`? No, keys are `AABA...`. 3. Match `A` then `A`. Reach node splitting `B` vs `C`. 4. Pattern `AAC...` matches `C` branch. 5. Descend to leaf, e.g., `AACAAAC`. 6. Verify LCP(`AACBACD`, `AACAAAC`). * `AAC` match. * `B` vs `A`. Mismatch. * `B > A`. * Lexicographic pos: After `AACAAAC`, before `BAB...`. Q2: Patricia Trie & Search|Back to Question

Sol 16/01/23 Q3

MOPHF Construction: * $h_1 = x + y, h_2 = x + 2y \pmod 7$. * $A = 1, B = 2, C = 3$. * **AA:** $h_1 = 2, h_2 = 3$. Edge (2,3). Rank 0. * **AC:** $h_1 = 4, h_2 = 0$. Edge (4,0). Rank 1. * **BB:** $h_1 = 4, h_2 = 6$. Edge (4,6). Rank 2. * **CC:** $h_1 = 6, h_2 = 2$. Edge (6,2). Rank 3. * **Graph:** $2 - 3, 4 - 0, 4 - 6, 6 - 2$. Cycle? * Path: $0 - 4 - 6 - 2 - 3$. No cycle. It is a line. * **Solve:** * $g[0] = 0$. * Eq(AC): $g[4] + 0 = 1 \to g[4] = 1$. * Eq(BB): $1 + g[6] = 2 \to g[6] = 1$. * Eq(CC): $1 + g[2] = 3 \to g[2] = 2$. * Eq(AA): $2 + g[3] = 0 \pmod 4 \to g[3] = 2$. Q3: Minimal Ordered Perfect Hash|Back to Question

---

Solutions: 02 February 2022

Sol 02/02/22 Q1

Text: `abababc`. **1. LZ77 (Triples):** * `a`: $\langle 0, 0, \text{'a'} \rangle$. * `b`: $\langle 0, 0, \text{'b'} \rangle$. * `aba`: Match `ab` at dist 2. $\langle 2, 2, \text{'a'} \rangle$. * `bc`: Match `b` at dist 4 (or 2). $\langle 2, 1, \text{'c'} \rangle$.

2. LZ78 (Trie): * Dict $D = \{\epsilon\}$. * `a`: Not in D. Add entry 1: `a`. Out $\langle 0, \text{'a'} \rangle$. * `b`: Not in D. Add entry 2: `b`. Out $\langle 0, \text{'b'} \rangle$. * `ab`: `a` is 1. `ab` not in. Add entry 3: `ab`. Out $\langle 1, \text{'b'} \rangle$. * `a`: In D (1). * `ab`: In D (3). * `abc`: Not in. Add entry 4: `abc`. Out $\langle 3, \text{'c'} \rangle$. Q1: LZ77 and LZ78|Back to Question

Sol 02/02/22 Q2

Pipeline `amata$`: 1. **BWT:** * Matrix sort $\to$ `L = atmaa$a`. (Check trace in notes). 2. **MTF:** * List `a, m, t`. * `a` (idx 0). * `t` (idx 2). List `t, a, m`. * `m` (idx 2). 3. **RLE0:** * Compress runs of 0s. 4. **Huffman:** * Build tree on frequencies. Q2: Full Compression Pipeline|Back to Question

Sol 02/02/22 Q3

1. Mutual Partitioning: * $L_1 = (1, 2, 4, 6, 9, 10, 15, 18, 20)$. * $L_2 = (2, 3, 7, 8, 18)$. * Pivot $m_2 = 7$ (median of L_2). * Search 7 in L_1. Split at 6|9. * Recurse Left: $(1, 2, 4, 6) \cap (2, 3)$. * Pivot 2. Match 2. Output 2. * Recurse Right: $(9 \ldots 20) \cap (8, 18)$. * Pivot 8. * Output 18.

2. Two-Level ($b = 3$): * Blocks L_1: $B_1(1, 2, 4), B_2(6, 9, 10), B_3(15, 18, 20)$. * Meta $L'_1 = (1, 6, 15)$. * Merge L'_1 with L_2: * 2 fits B_1. Scan B_1. Found 2. * 3 fits B_1. Scan. Not found. * 7 fits B_2. Scan. Not found. * 18 fits B_3. Scan. Found 18. Q3: Intersection Algorithms|Back to Question

---

Exam: 04 July 2022

Q1: Dense Code

Problem: Given the integer **6**, show how the **(s,c)-dense code** with parameters $s = 3$ and $c = 5$ encodes it. *(Hint: Derive first the number of bits used).* Sol 04/07/22 Q1|See Solution

Q2: BWT Pipeline

Problem: Given the string $S =$ "cbababaa": 1. Compute **BWT(S)**. 2. Apply **Move-To-Front (MTF)** to the BWT result. Initial list (a, b, c). Positions start from 0. 3. Apply **Huffman** to the MTF integers. Sol 04/07/22 Q2|See Solution

Q3: MOPHF Construction

Problem: Construct a Minimal Ordered Perfect Hash for $S = \{$aba, abb, bbb, caa, cba$\}$. * **Table size:** $m = 7$. * **Hash Functions:** * $h_1(X) = \sum \text{code}(X[i]) \pmod 7$ * $h_2(X) = \prod \text{code}(X[i]) \pmod 7$ * Codes: $a = 1, b = 2, c = 3$. Sol 04/07/22 Q3|See Solution

Q4: Interpolation Search

Problem: 1. Describe the Interpolation Search structure for $S = \{2, 3, 4, 9, 10, 18, 20, 21, 28, 30, 32, 36\}$. 2. Show the steps to search for key $y = 31$. Sol 04/07/22 Q4|See Solution

---

Exam: 05 June 2023

Q1: RLE0 Pipeline

Problem: String $S =$ "abababc". Apply **BWT** $\rightarrow$ **MTF** $\rightarrow$ **RLE0** (Wheeler) $\rightarrow$ **Huffman**. Sol 05/06/23 Q1|See Solution

Q2: Integer Compression

Problem: Sequence $S = (11, 14, 16, 19, 20, 21, 22)$. 1. Encode using **Elias-Fano**. 2. Encode using **Interpolative Coding** (just one level of recursion, first 3 numbers). Sol 05/06/23 Q2|See Solution

Q3: Treap Merge

Problem: 1. Construct **Max-Treap** T_1 with $\{\langle A, 8\rangle, \langle B, 2\rangle, \langle C, 9\rangle, \langle D, 4\rangle\}$. 2. Construct **Max-Treap** T_2 with $\{\langle H, 3\rangle, \langle M, 7\rangle, \langle G, 0\rangle, \langle L, 1\rangle\}$. 3. Show the result of **Merge**(T_1, T_2). Sol 05/06/23 Q3|See Solution

---

Exam: 07 September 2023

Q1: Snow Plow Trace

Problem: Simulate **Snow Plow** on $S = (2, 6, 5, 3, 1, 7, 2)$ with memory $M = 2$. Show the sorted blocks. Sol 07/09/23 Q1|See Solution

Q2: Patricia Trie

Problem: Ordered set $S = \{$AABA, AACAAAC, AACAACC, BABAA, BABBB, BACA$\}$. 1. Build the **Patricia Trie**. 2. Show lexicographic search for $P =$ AACBACD. Sol 07/09/23 Q2|See Solution

Q3: Rank Structure

Problem: Binary array $B = [0\ 0\ 0\ 1\ 1\ 1\ 0\ 0\ 0\ 1]$. Build the **Rank** data structure ($O(1)$) assuming: * Big block size $Z = 4$. * Small block size $z = 2$. Sol 07/09/23 Q3|See Solution

Solutions

Solutions: 04 July 2022

Sol 04/07/22 Q1

(s,c)-dense code: * $s = 3$ (Stoppers), $c = 5$ (Continuers). * Sum $s + c = 8 = 2^3$. Codewords are 3 bits long. * **Stoppers** ($0 \ldots 2$)**:** `000, 001, 010`. * **Continuers** ($3 \ldots 7$)**:** `011, 100, 101, 110, 111`. * **Ranges:** * 1 byte (1 chunk): Values $0 \ldots 2$ (3 values). * 2 bytes (2 chunks): $5 \times 3 + 0 \ldots 5 \times 3 + 2$ ($3 \ldots 17$). (Wait, logic is $Val = (k$ continuers$) \times s +$ stopper?) * Let's list them: * 0: `000` * 1: `001` * 2: `010` * 3: `011 000` (Cont 0 + Stop 0) * 4: `011 001` * 5: `011 010` * **6:** `100 000` (Cont 1 + Stop 0). * **Answer:** `100 000`. Q1: Dense Code|Back to Question

Sol 04/07/22 Q2

1. BWT(cbababaa): * Rotations: `cbababaa`, `a...`, `aa...`, `aba...`, etc. * Sorted Matrix M: 1. `aa...` (L: `b`) 2. `abaa...` (L: `b`) 3. `ababaa...` (L: `b`) 4. `bababaa...` (L: `c`) 5. `babaa...` (L: `a`) 6. `baa...` (L: `a`) 7. `cbababaa` (L: `a`) 8. ... * **L (BWT):** `bbbc aaa a` (Need to trace exact sort). * Let's sort suffixes/rotations of `cbababaa`: * `a` (7) $\rightarrow$ pred `a` * `aa` (6) $\rightarrow$ pred `b` * `abaa` (4) $\rightarrow$ pred `b` * `ababaa` (2) $\rightarrow$ pred `b` * `bababaa` (1) $\rightarrow$ pred `c` * `babaa` (3) $\rightarrow$ pred `a` * `baa` (5) $\rightarrow$ pred `a` * `cbababaa` (0) $\rightarrow$ pred `a` * Sorted: `a`, `aa`, `abaa`, `ababaa`, `baa`, `babaa`, `bababaa`, `c...`. * L column: `a, b, b, b, a, a, c, a`. (Wait, char preceding `a` at 7 is `a` at 6. Preceding `aa` at 6 is `b` at 5). * Result: `a b b b a a c a`? (Verify).

2. MTF: * Init: `a, b, c`. * Input `a` (idx 0) $\rightarrow$ Out `0`. List `a, b, c`. * Input `b` (idx 1) $\rightarrow$ Out `1`. List `b, a, c`. * Input `b` (idx 0) $\rightarrow$ Out `0`. * ...

3. Huffman: * Frequencies of MTF output integers. Q2: BWT Pipeline|Back to Question

Sol 04/07/22 Q3

MOPHF Construction: * S: `aba, abb, bbb, caa, cba`. * Rank: 0, 1, 2, 3, 4. * Codes: $a = 1, b = 2, c = 3$. * $h_1 = \sum, h_2 = \prod \pmod 7$. * **aba:** $h_1 = 1 + 2 + 1 = 4, h_2 = 2$. Edge (4,2), rank 0. * **abb:** $h_1 = 1 + 2 + 2 = 5, h_2 = 4$. Edge (5,4), rank 1. * **bbb:** $h_1 = 6, h_2 = 8 \equiv 1$. Edge (6,1), rank 2. * **caa:** $h_1 = 3+1+1 = 5, h_2 = 3$. Edge (5,3), rank 3. * **cba:** $h_1 = 3+2+1 = 6, h_2 = 6$. Edge (6,6), rank 4. * **Graph:** (4,2), (5,4), (6,1), (5,3). Loop (6,6). * **Values (g):** * Loop (6,6) with value 4: $g[6] + g[6] = 4 \pmod 5$? No, mod is usually $N = 5$. * $2g[6] \equiv 4 \pmod 5 \implies g[6] = 2$. * Edge (6,1) rank 2: $2 + g[1] = 2 \implies g[1] = 0$. * ... Solve rest. Q3: MOPHF Construction|Back to Question

Sol 04/07/22 Q4

Interpolation Buckets: * S size 12. Range $2 \ldots 36$. * $b = (36 - 2 + 1)/12 = 35/12 \approx 2.9 \rightarrow 3$. * Buckets: $B_1[2..4], B_2[5..7] \ldots$ * **Search 31:** * $j = \lfloor (31 - 2)/3 \rfloor + 1 = \lfloor 29/3 \rfloor + 1 = 9 + 1 = 10$. * Check bucket 10. Range $[2 + 9 \times 3 \ldots] = [29, 31]$. * Bucket 10 content: `30`? ($S[10]$ is 30?). * Binary search in Bucket 10. Q4: Interpolation Search|Back to Question

---

Solutions: 05 June 2023

Sol 05/06/23 Q1

Pipeline `abababc`: 1. **BWT:** `L = cbbbaaa`. (Last col of sorted rotations). 2. **MTF:** Init `a, b, c`. * `c` (idx 2) → 2. List `c, a, b`. * `b` (idx 2) → 2. List `b, c, a`. * `b` (idx 0) → 0. * `b` (idx 0) → 0. * `a` (idx 2) → 2. List `a, b, c`. * ... Output `2, 2, 0, 0, 2, 0, 0`. 3. **RLE0:** * `2` → 3. * `2` → 3. * `0, 0` → Run len 2. Wheeler `10` (2+1=3 bin `11` drop 1 → `1`? No, `1` → `0`, `2` → `1`...). * Wheeler: $x \to x+1$ in binary, remove MSB. * Run 2 → 3(11) → 1. * Stream: `3, 3, 1, ...` Q1: RLE0 Pipeline|Back to Question

Sol 05/06/23 Q2

1. Elias-Fano: * $S = 11, 14, 16, 19, 20, 21, 22$. $n = 7$. $u = 23$. * $l = \lfloor \log(23/7) \rfloor = 1$. * Encode low (1 bit) and high (unary gaps).

2. Interpolative (3 numbers): * Range indices $[1, 7]$. Low 11, High 22. * $m = 4$. $S[4] = 19$. * Bounds: $11 + (4 - 1) = 14$. $22 - (7 - 4) = 19$. * Encode 19 in $[14, 19]$. * Recurse left $[1, 3]$. Recurse right $[5, 7]$. Q2: Integer Compression|Back to Question

Sol 05/06/23 Q3

1. Max-Treap T1: * Pairs: (A,8), (B,2), (C,9), (D,4). * Root max prio: **C(9)**. * Left: $\{A, B\}$. Max A(8). Root **A**. Right **B(2)**. * Right: $\{D\}$. Root **D(4)**. * Structure: $C \to (L)A \to (R)B, (R)D$.

2. Max-Treap T2: * Pairs: (H,3), (M,7), (G,0), (L,1). * Root max prio: **M(7)**. * Left: H, G, L. Max H(3). Root **H**. * Left of H: $G(0)$. * Right of H: $L(1)$. * Structure: $M \to (L)H \to (L)G, (R)L$.

3. Merge: * Join T_1 (keys A-D) and T_2 (keys G-M). * Dummy root $(-\infty)$. T_1 Left, T_2 Right. * Sink dummy? No, max heap. Dummy should be $+\infty$. * Sink $+\infty$ down. Q3: Treap Merge|Back to Question

---

Solutions: 07 September 2023

Sol 07/09/23 Q1

Snow Plow (M=2): 1. Load `2, 6`. $H = [2, 6]$. 2. Out `2`. Read `5`. $5 \geq 2 \to H = [5, 6]$. 3. Out `5`. Read `3`. $3 < 5 \to U = [3]$. $H = [6]$. 4. Out `6`. Read `1`. $1 < 6 \to U = [3, 1]$. $H = []$. * **Run 1:** `2, 5, 6`. * Load $U \to H = [1, 3]$. 5. Out `1`. Read `7`. $H = [3, 7]$. 6. Out `3`. Read `2`. $U = [2]$. $H = [7]$. 7. Out `7`. * **Run 2:** `1, 3, 7`. Q1: Snow Plow Trace|Back to Question

Sol 07/09/23 Q3

Rank Structure: * $B = 0001110001$. Len 10. * $Z = 4$. Superblocks at 0, 4, 8. * $S_0 = 0$. * $S_1 = \text{pop}(0001) = 1$. * $S_2 = 1 + \text{pop}(1100) = 1 + 2 = 3$. * $z = 2$. Blocks inside SB. * SB0: `00` (0), `01` (0). * SB1: `11` (0, relative), `00` (2). * **Rank(9):** (Index 9 is last bit? 1-based: 10th bit). * SB index 2 ($i = 8$). Base 3. * Block offset 0 ($8 \ldots 9$). * Bits `01`. Pop `1`. * Total $3 + 1 = 4$. Q3: Rank Structure|Back to Question

Exercises

This document contains exercises derived from the course lectures and notes, organized by topic. Each exercise links to its detailed solution at the end of the file.

Part I: Sorting & I/O

Ex 1.1: Snow Plow Trace

Problem: Trace the run generation using the **Snow Plow** algorithm. * **Memory:** $M = 2$ (Assume memory is split: 1 slot for the Min-Heap H, 1 slot for the Unsorted Buffer U initially, or conceptualize it as a heap of size 2 that shrinks). * **Input Sequence:** $S = [1, 7, 5, 3, 2, 1, 0]$. Show the content of the Heap, the Unsorted buffer, and the Output Runs step-by-step. Sol 1.1 (Snow Plow)|See Solution

---

Part II: Sets & Search

Ex 2.1: Interpolation Search Buckets

Problem: We have a sorted array X of $n = 12$ integers:

$$X = [1, 2, 3, 8, 9, 17, 19, 20, 28, 30, 32, 36]$$

1. Calculate the bucket size b. 2. List the content of the non-empty buckets ($B_1 \dots B_{12}$). 3. Define the ID array I (start/end positions). Sol 2.1 (Interpolation Buckets)|See Solution

---

Part III: Advanced Data Structures

Ex 3.1: Treap Construction and Split

Problem: 1. Build a **Min-Treap** (inserting node by node) with the following pairs of $\langle \text{Key}, \text{Priority} \rangle$:

$$S = [\langle L, 5\rangle, \langle M, 3\rangle, \langle Z, 10\rangle, \langle D, 2\rangle, \langle A, 4\rangle]$$

2. Perform a **Split** operation on the resulting tree using key $k = F$. Sol 3.1 (Treaps)|See Solution

---

Part IV: Indexing & Hashing

Ex 4.1: Multi-key Quicksort Trace

Problem: Sort the following set of strings using Multi-key Quicksort (3-way string quicksort).

$$S = \{\text{"cat", "abi", "cast", "car", "at"}\}$$

Assumption: Always pick the **first string** in the current set as the pivot. Sol 4.1 (Multi-key Quicksort)|See Solution

Ex 4.2: Cuckoo Hashing Insertion

Problem: Insert the sequence $S = [1, 5, 8, 3, 12, 10, 11, 13, 9, 15]$ into two Cuckoo tables T_1, T_2 of size 7. **Hash Functions:** * $h_1(k) = k \pmod 7$ * $h_2(k) = 3k \pmod 7$ Show the state of the tables and indicate if a Rehash is required. Sol 4.2 (Cuckoo Hashing)|See Solution

---

Part V: Data Compression

Ex 5.1: Integer Encoding

Problem: Given the integer sequence $S = [1, 6, 15, 18, 21, 24, 30]$. 1. Transform it into a **Gap Sequence** (S'). 2. Encode the first 3 numbers of S' using **Elias Gamma**. 3. Encode the first 3 numbers of S' using **Elias Delta**. 4. Encode the first 3 numbers of S' using **Rice Coding** with $k = 3$. Sol 5.1 (Integer Encoding)|See Solution

Ex 5.2: PForDelta Encoding

Problem: Encode the sequence $S = [1, 5, 9, 3, 3, 6]$ using PForDelta. **Parameters:** $b = 3$ bits, $base = 1$. Use `111` as the Escape symbol. Show the bitstream and the Exception Array. Sol 5.2 (PForDelta)|See Solution

Ex 5.3: Integer Decoding

Problem: You receive the following bitstream: `00110100010` 1. Decode it assuming it is **Elias Gamma**. 2. Decode it assuming it is **Elias Delta**. Sol 5.3 (Decoding)|See Solution

Ex 5.4: LZ77 vs LZSS Encoding

Problem: Encode the string $T =$ "aababc" using: 1. **LZ77:** Using triples $\langle d, l, c \rangle$. 2. **LZSS:** Using flags (`0` for literal, `1` for copy $\langle d, l \rangle$). Sol 5.4 (LZ77 vs LZSS)|See Solution

Solutions

Sol 1.1 (Snow Plow)

Initial State: Memory size 2. 1. **Load:** Read `1, 7`. Heap $H = [1, 7]$. Unsorted $U = []$. 2. **Output 1:** Min `1`. Output `[1]`. * **Read 5:** $5 \geq 1$. Insert into H. $H = [5, 7]$. 3. **Output 5:** Min `5`. Output `[1, 5]`. * **Read 3:** $3 < 5$. Cannot fit in run. Insert into U. $H = [7]$, $U = [3]$. 4. **Output 7:** Min `7`. Output `[1, 5, 7]`. * **Read 2:** $2 < 7$. Insert into U. $H = []$, $U = [3, 2]$. * **Heap Empty:** End of Run 1. * **Restart:** Move U to H. $H = [2, 3]$. 5. **Output 2:** Min `2`. Output `[2]`. * **Read 1:** $1 < 2$. Insert into U. $H = [3]$, $U = [1]$. 6. **Output 3:** Min `3`. Output `[2, 3]`. * **Read 0:** $0 < 3$. Insert into U. $H = []$, $U = [1, 0]$. * **Heap Empty:** End of Run 2.

Final Runs: `[1, 5, 7]` and `[2, 3]`. Ex 1.1: Snow Plow Trace|Back to Question

---

Sol 2.1 (Interpolation Buckets)

1. **Bucket Size:** $b = \frac{X_n - X_1 + 1}{n} = \frac{36-1+1}{12} = \frac{36}{12} = 3$.
2. **Buckets:** Range is $[1, 36]$. Each bucket covers range of 3 values.
   - $B_1[1..3] : \{1, 2, 3\}$
   - $B_3[7..9] : \{8, 9\}$
   - $B_7[19..21] : \{19, 20\}$
   - $B_{10}[28..30] : \{28, 30\}$
   - $B_{11}[31..33] : \{32\}$
   - $B_{12}[34..36] : \{36\}$
   - *(Buckets 2, 4, 5, 6, 8, 9 are empty).*
3. **ID Array:**
   - $I[1] = \langle 1, 3 \rangle$
   - $I[3] = \langle 4, 5 \rangle$
   - $I[7] = \langle 6, 7 \rangle$
   - $I[10] = \langle 8, 9 \rangle$
   - $I[11] = \langle 10, 10 \rangle$
   - $I[12] = \langle 11, 11 \rangle$
   - Others: $\langle -1, -1 \rangle$ Ex 2.1: Interpolation Search Buckets|Back to Question

---

Sol 3.1 (Treaps)

1. Construction: * **Insert (L, 5):** Root is $(L, 5)$. * **Insert (M, 3):** $M > L$ (Right). Priority $3 < 5 \rightarrow$ Rotate M up. **Root: M**. Left child: L. * **Insert (Z, 10):** $Z > M$. Priority $10 > 3 \rightarrow$ Right child of M. *

Insert (D, 2): $D < M$ (Left). $D < L$ (Left of L). Priority $2 < 5$ and $2 < 3 \rightarrow$ Rotate D up over L, then over M. **Root: D**. Right child: M. * **Insert (A, 4):** $A < D$ (Left). Priority $4 > 2 \rightarrow$ Left child of D.

Final Tree:

```
graph TD
    D((D, 2)) --> A((A, 4))
    D --> M((M, 3))
    M --> L((L, 5))
    M --> Z((Z, 10))
```

2. Split on F:

- Goal: $T_{\leq F}$ and $T_{>F}$.
- Insert phantom node $(F, -\infty)$.
- $F > D$ (Right). $F < M$ (Left). $F < L$ (Left).
- F has minimal priority $(-\infty)$, rotates up to become the new Root.
  - L becomes right child of F.
  - M becomes parent of F's right subtree? No, M is higher priority.
  - Rotations bring F to root.
- **Result:**
  - Left Tree ($T_{\leq F}$): Root **D**, Left **A**.
  - Right Tree ($T_{>F}$): Root M, Left L, Right Z.

Ex 3.1: Treap Construction and Split|Back to Question

Sol 4.1 (Multi-key Quicksort)

Set: `{"cat", "abi", "cast", "car", "at"}`. $i = 0$.

1. **Pivot:** "cat" ($p[0] =$ 'c').
   - $S_<$('a' < 'c'): `{"abi", "at"}`.
   - $S_=$: `{"cat", "cast", "car"}`.
   - $S_>$: $\emptyset$.
2. **Recurse on $S_<$ (index 0):** `{"abi", "at"}`. Pivot "abi" ('a').
   - $S_=$: `{"abi", "at"}` (Match 'a'). Recurse index **1**.
   - Pivot "abi" ($p[1] =$ 'b').
   - $S_>$: `{"at"}` ('t' > 'b').
   - **Result:** `abi` (from $S_=$), `at` (from $S_>$). Order: `abi, at`.
3. **Recurse on $S_=$ (index 1):** `{"cat", "cast", "car"}`. Pivot "cat" ('a').
   - $S_=$: All match. Recurse index **2**.
   - Pivot "cat" ($p[2] =$ 't').
   - $S_<$('r' < 't'): `{"car"}`.
   - $S_<$('s' < 't'): `{"cast"}`. Note: 's'(115) < 't'(116).

- $S_=$: `{"cat"}`.
- **Sub-recurse on $S_<$:** `{"car", "cast"}`. Index 2. Pivot "car". 'r' vs 's'. 'r' is pivot. 's' goes to $S_>$.
- **Result:** `car, cast`.

4. Concatenate: abi, at + car, cast, cat.

Ex 4.1: Multi-key Quicksort Trace|Back to Question

Sol 4.2 (Cuckoo Hashing)

Setup: T_1, T_2 size 7. $h_1(k) = k\%7, h_2(k) = 3k\%7$.

1. **1:** $T_1[1] = 1$.
2. **5:** $T_1[5] = 5$.
3. **8:** $h_1(8) = 1$. Clash with 1. **Kick 1** $\rightarrow T_1[1] = 8$.
   - Reinsert 1: $h_2(1) = 3$. $T_2[3] = 1$.
4. **3:** $T_1[3] = 3$.
5. **12:** $h_1(12) = 5$. Clash with 5. **Kick 5** $\rightarrow T_1[5] = 12$.
   - Reinsert 5: $h_2(5) = 15\%7 = 1$. $T_2[1] = 5$.
6. **10:** $h_1(10) = 3$. Clash 3. **Kick 3** $\rightarrow T_1[3] = 10$.
   - Reinsert 3: $h_2(3) = 9\%7 = 2$. $T_2[2] = 3$.
7. **11:** $T_1[4] = 11$.
8. **13:** $T_1[6] = 13$.
9. **9:** $T_1[2] = 9$.
10. **15:** $h_1(15) = 1$. Clash 8. **Kick 8** $\rightarrow T_1[1] = 15$.
    - Reinsert 8: $h_2(8) = 3$. Clash 1. **Kick 1** $\rightarrow T_2[3] = 8$.
    - Reinsert 1: $h_1(1) = 1$. Clash 15. **Kick 15** $\rightarrow T_1[1] = 1$.
    - Reinsert 15: $h_2(15) = 3$. Clash 8. **Kick 8**.
    - CYCLE DETECTED (1 and 15 kicking each other at $T_1[1]$ and $T_2[3]$).

      Result: Insertion of 15 triggers a Rehash.

      Ex 4.2: Cuckoo Hashing Insertion|Back to Question

Sol 5.1 (Integer Encoding)

Gap Sequence: $S' = [1, 5, 9, 3, 3, 3, 6]$. (Diffs: $6 - 1 = 5, 15 - 6 = 9 \ldots$).

1. **Elias Gamma:** $\underbrace{0 \ldots 0}_{\lfloor \log x \rfloor} \text{bin}(x)$
   - $1 \rightarrow 1$
   - $5 \rightarrow 00101$ (bin 101, len 3, 2 zeros)
   - $9 \rightarrow 0001001$ (bin 1001, len 4, 3 zeros)

2. **Elias Delta:** $\gamma(\text{len}) \cdot \text{suffix}$
   - 1: len 1 $\to \gamma(1) = 1 \to 1$.
   - 5: len 3 $\to \gamma(3) = 011$. Suffix of 5 (`101`) is `01`. Result: 01101.
   - 9: len 4 $\to \gamma(4) = 00100$. Suffix `001`. Result: 00100001.
3. **Rice ($k = 3$):** $q = \lfloor (x-1)/8 \rfloor$, $r = (x-1)\%8$. Format: $U(q) \cdot \text{bin}_3(r)$.
   - 1: $q = 0, r = 0 \to 1 \cdot 000$.
   - 5: $q = 0, r = 4 \to 1 \cdot 100$.
   - 9: $q = 1, r = 0 \to 01 \cdot 000$ (assuming $U(q)$ is $0^q 1$).

   Ex 5.1: Integer Encoding|Back to Question

Sol 5.2 (PForDelta)

Range: $[1, 1 + 2^3 - 2] = [1, 7]$. Encode $x - 1$. Exception if $x > 7$.

Sequence: $1, 5, 9, 3, 3, 6$.

- **1:** $1 - 1 = 0 \to 000$.
- **5:** $5 - 1 = 4 \to 100$.
- **9:** $9 > 7$. **Exception.** Emit Escape `111`. Add 9 to Extra.
- **3:** $3 - 1 = 2 \to 010$.
- **3:** 010.
- **6:** $6 - 1 = 5 \to 101$.

Bitstream: 000 100 111 010 010 101

Exception Array: [9]

Ex 5.2: PForDelta Encoding|Back to Question

Sol 5.3 (Decoding)

Stream: `00110100010`

1. Gamma Decoding ($0^N 1 \ldots$):

- `001`: $N = 2$. Read 2 more bits `10`. Value $110_2 = 6$.
- Remaining: `100010`.
- `1`: $N = 0$. Read 0 bits. Value $1_2 = 1$.
- Remaining: `00010`.
- `0001`: $N = 3$. Read 3 more bits `0`... Stream ends! **Error/Incomplete**.

2. Delta Decoding ($\gamma(\text{len}) \ldots$):

- `001`: γ code. $N = 2 \to 11 = 3$. Length is 3.
- Read 3 bits (without MSB? No, standard Delta reads suffix).
- Let's assume standard: $\gamma(3) = 011$.

- Stream starts `001`? No, $\gamma(3)$ is `011`.
- Let's re-parse `001` as γ. `001` is $\gamma(100_2 = 4)$.
- Len is 4. Read 3 suffix bits: `101`.
- Value: `1101` = 13.
- Remaining: `00010`.
- Next γ: `0001`. $N = 3 \to 1000_2 = 8$. Len is 8.
- Not enough bits.

Alternative parsing (Start with `00110...`):

- `00110`: Gamma prefix `001` ($N = 2 \to 11 = 3$).
- Len 3. Suffix `01`. Value $101_2 = 5$.
- Remaining: `00010`.
- `0001`: Gamma prefix ($N = 3 \to 100_2 = 4$).
- Len 4. Suffix `010`. Value $1010_2 = 10$.
- Result: 5, 10. (This fits perfectly).

Ex 5.3: Integer Decoding|Back to Question

Sol 5.4 (LZ77 vs LZSS)

Text: aababc

LZ77 (Triples $\langle d, l, c \rangle$):

1. `a`: No match. $\langle 0, 0, \text{'a'} \rangle$.
2. `a`: Match `a` at dist 1. $\langle 1, 1, \text{'b'} \rangle$. (Encodes `ab`)
3. `abc`: Match `ab` at dist 2. $\langle 2, 2, \text{'c'} \rangle$. (Encodes `abc`)
   - *Note: LZ77 always consumes the char after the match.*

LZSS (Flags + Pairs):

1. `a`: No match. Literal `0 a`.
2. `a`: Match `a` (len 1). Is it worth it? (Usually min len ≥ 2). If we encode: `1` $\langle 1, 1 \rangle$.
3. `b`: Literal `0 b`.
4. `ab`: Match `ab` (len 2). `1` $\langle 2, 2 \rangle$.
5. c: Literal 0 c.

Ex 5.4: LZ77 vs LZSS Encoding|Back to Question